

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ» (ТУСУР)

Кафедра комплексной информационной безопасности электронно-
вычислительных систем (КИБЭВС)

Автоматизированная система «DreamBreaker»

Курсовая работа по дисциплине «Технологии и методы программирования»

Пояснительная записка

Студенты гр. 723-1

_____ М.А. Быстров

_____ Н.Г. Праскурин

«__» _____ 2026

Руководитель

Старший преподаватель

кафедры КИБЭВС

_____ _____ Н.С. Репьюк

оценка

«__» _____ 2026

Реферат

Курсовая работа содержит 92 страницы пояснительной записки, 48 рисунков, 8 источников.

ЯЗЫК ПРОГРАММИРОВАНИЯ RUST, VISUAL STUDIO, БАЗЫ ДАННЫХ, POSTGRESQL, PGADMIN 4, CARGO, ICED.

Программа: «DreamBreaker».

Цель работы: необходимо предоставить графический интерфейс пользователя для авторизации, регистрации и смены пароля, спроектировать и разработать настольную стратегическую и экономическую игру «DreamBreaker» на основе механик Монополии, со следующими функциями: авторизация, регистрация, выход из аккаунта, создание и загрузка игровых партий, перемещение по игровому полю (40 клеток, 4 стороны), покупка и улучшение собственности, выплата ренты и налогов, взаимодействие с магазином усилений (реролл, покупка усилений), управление инвентарём, ход ботов-противников, просмотр статистики профиля, сохранение и возобновление партии, сдача. Разработка программы проводилась на языке программирования RUST.

Курсовая работа выполнена в текстовом редакторе Microsoft Word 2016.

Пояснительная записка оформлена согласно ОС ТУСУР 01–2021. [1].

Abstract

The coursework contains 92 pages of the explanatory note, 48 figures, 8 sources.

RUST PROGRAMMING LANGUAGE, VISUAL STUDIO, DATABASES, POSTGRESQL, PGADMIN 4, CARGO, ICED.

Program: "DreamBreaker".

Purpose of the work: to provide a graphical user interface for authentication, registration, and password change; to design and develop the board strategy and economic game "DreamBreaker" based on Monopoly mechanics, with the following features: authentication, registration, logout, creating and loading game sessions, movement across the game board (40 cells, 4 sides), purchasing and upgrading properties, paying rent and taxes, interaction with the power-up shop (reroll, purchasing power-ups), inventory management, bot opponent turns, viewing profile statistics, saving and resuming a session, and surrendering.

The program was developed using the RUST programming language.

The course work was completed in Microsoft Word 2016.

The explanatory note is formatted in accordance with OS TUSUR 01–2021.

[1].

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное бюджетное образовательное
учреждение высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра комплексной информационной безопасности электронно-
вычислительных систем (КИБЭВС)

УТВЕРЖДАЮ

Зав. кафедрой КИБЭВС,

д.т.н., профессор

_____ А.А. Шелупанов

(подпись)(Ф.И.О.)

« _____ » _____ 20__ г.

ЗАДАНИЕ

на курсовую работу по дисциплине «Технологии и методы программирования»
студентам группы 723-1 факультета безопасности Быстрову Михаилу
Алексеевичу, Праскурину Никите Георгиевичу.

1. Тема работы: «Экономическая и стратегическая игра»
2. Цель работы: Создание программной среды для пользователей, имитирующей обусловленные игровыми условиями экономические процессы.
3. Индивидуальные задачи: Праскурин Никита Георгиевич:
разработка технического задания, проведение оценки рисков, разработка элементов графического интерфейса, разработка базы данных, разработка и интеграция серверной части;

Быстров Михаил Алексеевич: создание диаграмм, разработка клиентской части, создание функциональной модели системы, разработка и проведение тестирования, разработка и интеграция серверной части.

4. Дата выдачи задания: « ____ » _____ 202__ г.

5. Дата сдачи работы: « ____ » _____ 202__ г.

Задание выдал

Ст. препод. каф. КИБЭВС, Репьюк Н.С. _____ « ____ » _____.202__.

(Должность, Ф.И.О.)

(Подпись)

(дата)

Задание приняли к исполнению:

_____ « ____ » _____.202__.

(Ф.И.О.)

(Подпись)

(дата)

_____ « ____ » _____.202__.

(Ф.И.О.)

(Подпись)

(дата)

Оглавление

Оглавление	6
Введение	8
1 ОЦЕНКА РИСКОВ ПРОЕКТА	9
1.1 Оценка рисков для построения информационной системы	9
1.2 Меры по предотвращению наиболее возможных рисков	11
2 РАЗРАБОТКА СТРУКТУРНОЙ, ФУНКЦИОНАЛЬНОЙ И ИНФОРМАЦИОННОЙ МОДЕЛЕЙ	15
2.1 Описание процессов системы в методологии функционального моделирования IDEF0	15
2.2 Информационная модель базы данных	18
2.3 Структурная модель системы.....	21
2.4 Информационная модель системы	22
3 РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ	24
3.1 Создание таблиц	24
3.2 Реализация триггеров и хранимых функций	28
4 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ	30
4.1 Архитектура серверной части	30
4.2 Аутентификация пользователей	31
4.3 Работа с данными и бизнес-логика.....	32
5 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ.....	35
5.1 Внутреннее устройство клиентского приложения	35
5.2 Интерфейс системы	36
6 ТЕСТИРОВАНИЕ И ДОРАБОТКА СИСТЕМЫ.....	51
6.1 Ручное тестирование.....	51
6.2 Функциональное тестирование.....	55
7 ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ	59
Заключение	60
Список использованных источников	62
Приложение А (Обязательное) Оценка рисков Репьюк Н. С.....	63

Приложение Б (Обязательное) Оценка рисков Мацкевич Е. В.....	67
Приложение В (Обязательное) Оценка рисков Мальчукова А. Н.....	71
Приложение Г (Обязательное) Диаграммы и декомпозиции диаграмм..	75
Приложение Д (Обязательное) Ссылки на репозитории	82
Приложение Е (Обязательное) Результаты ручного тестирования	83

Введение

Целью курсовой работы является проектирование и разработка автоматизированной системы «DreamBreaker», предназначенной для предоставления пользователям интерактивной среды в виде стратегической и экономической игры. Разрабатываемая система должна обеспечивать функции регистрации и авторизации пользователей, создания, хранения и управления партиями, их параметрами и состояниями, подсчёта игровой валюты, сохранения и отображения игровой статистики, реализации искусственных соперников.

В ходе выполнения курсовой работы были реализованы следующие этапы:

1. Составление технического задания;
2. Проведение оценки рисков;
3. Разработка структурной, функциональной и информационной модели системы;
4. Проектирование базы данных;
5. Проектирование серверной части приложения;
6. Проектирование клиентской части приложения;
7. Разработка и интеграция серверной и клиентской частей;
8. Тестирование и доработка системы;
9. Написание руководства пользователя и технической документации;
10. Подготовка пояснительной записки и отчета о проделанной работе.

Курсовая работа направлена на закрепление теоретических знаний, полученных при изучении дисциплин, связанных с программированием, проектированием программных систем, базами данных и разработкой пользовательских интерфейсов. В ходе выполнения работы также формируются практические навыки создания веб-приложения, взаимодействующего с серверной частью и базой данных, а также навыки реализации разграничения прав доступа пользователей.

1 ОЦЕНКА РИСКОВ ПРОЕКТА

1.1 Оценка рисков для построения информационной системы

В ходе работы по оценке рисков были определены следующие потенциальные риски информационной системы:

1. Ошибки в работе продукта, не полная или не качественная реализация функционала;
2. Утрата результатов деятельности;
3. Неверное математическое обоснование;
4. Компрометация учетных данных пользователей;
5. Нестабильность и низкая производительность;
6. Использование программного продукта не по прямому назначению;
7. Компрометация базы данных;
8. Не санкционированный доступ(НСД) к базе данных;
9. Недоступность системы для легитимных пользователей;
10. Утрата игрового прогресса, подмена данных, неработоспособность приложения;
11. Нарушение игрового баланса, не соответствие ожидаемому поведению системы, сложность отладки;
12. Некорректная обработка специальных символов.

Выбранные риски основаны на анализе особенностей проектируемой системы. При их определении учитывались особенности структуры системы, особенности её работы, особенности работы функций и базы данных.

Для проведения оценки рисков, связанных с построением и работой системы, была организована экспертная оценка с участием трёх экспертов. На основании их суждений о вероятности возникновения и критичности последствий тех или иных рисков был рассчитан коэффициент

конкордации, отражающий степень согласованности мнений экспертов. В приложениях А, Б, В представлены индивидуальные оценки экспертов.

В таблице 1.1 представлена сводная таблица оценок вероятности возникновения рисков.

Таблица 1.1 – Оценка вероятности возникновения рисков

\ Эксперт №Риска \	Репьюк Н.С.	Мальчуков А.Н.	Мацкевич Е.В.
1	4	6	2
2	1	4	4
3	5	4	5
4	2	4	1
5	5	4	5
6	2	2	2
7	4	4	2
8	2	4	2
9	5	2	2
10	2	4	7
11	6	2	6
12	6	2	8

В таблице 1.2 представлена таблица оценки критичности последствий рисков.

Таблица 1.2 – Оценка критичности последствий возникновения рисков

\ Эксперт \ №Риска \	Репьюк Н.С.	Мальчуков А.Н.	Мацкевич Е.В.
1	2	8	7
2	6	6	8
3	4	6	9
4	7	4	5
5	3	2	3
6	1	2	3
7	7	6	10
8	6	4	7

Окончание таблицы 1.2

\ Эксперт \ №Риска \	Репьюк Н.С.	Мальчуков А.Н.	Мацкевич Е.В.
10	6	4	2
11	2	2	4
12	3	4	5

Произведен расчет значений коэффициента конкордации для каждой из таблиц.

Для таблицы 1.1 значение эффекта конкордации составило 0,0909, а для таблицы 1.2 составило 0,24697

Полученные значения показывают, что мнения экспертов не являются полностью совпадающими, однако достаточно совпадающими для возможности использования результатов для проведения предварительного анализа рисков. С учетом достаточного количества экспертов данные оценки могут быть использованы для дальнейшей классификации рисков и выбора мер по снижению вероятности их возникновения и уменьшения критичности их последствий.

1.2 Меры по предотвращению наиболее возможных рисков

Для классификации рисков была разработана матрицы оценки уровня риска, представленная на рисунке 1.1. Матрица позволяет определить уровень риска на основании показателей вероятности возникновения риска и критичности последствий его возникновения.

Вероятность	1	2	3	4	5	6	7	8	9	10
Критичность										
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	5	2	8	6	13	12	20	20	29	30
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

Рисунок 1.1 – Матрица оценки уровня риска

На основе полученных экспертных оценок риски были отсортированы в соответствии с уровнем риска, рассчитанным как произведение средней вероятности возникновения риска и средней степени возможных последствий.

В соответствии с используемой матрицей были приняты следующие уровни риска:

1. От 1 до 20 – низкий риск;
2. От 21 до 40 – умеренный риск;
3. От 41 до 60 – средний риск;
4. От 61 до 80 – высокий риск;
5. От 81 до 100 – Очень высокий риск.

В таблице 1.3 приведён список рисков, классификация и оценка уровня каждого риска.

Таблица 1.3 – классификация и оценка уровня рисков

Риск	Вероятность риска	Классификация риска
Ошибки в работе продукта, не полная или не качественная реализация функционала	22,7	умеренный
Утрата результатов деятельности	20	низкий
Неверное математическое обоснование	30	умеренный
Компрометация учётных данных пользователей	12,4	низкий
Нестабильность и низкая производительность приложения	12,4	низкий
Использование программного продукта не по прямому назначению	4	низкий
Компрометация базы данных	26	умеренный

Окончание таблицы 1.3

Риск	Вероятность риска	Классификация риска
НСД к системе	15	низкий
Недоступность системы для легитимных пользователей	13	низкий
Утрата игрового прогресса, подмена данных, неработоспособность приложения	17,3	низкий
Нарушение игрового баланса, несоответствие ожидаемому поведению системы, сложность отладки	12,4	низкий
Некорректная обработка специальных символов	21,3	умеренный

На основании полученных данных было принято решение о разработке и внедрении комплекса мероприятий, направленных на предотвращение или минимизацию последствий выявленных рисков. При этом риски умеренного уровня требуют повышенного внимания, так как имеют более высокие значения итоговой оценки, а риски низкого уровня контролируются за счёт базовых организационных и технических мер. В таблице 1.4 представлены меры предотвращения или уменьшения рисков.

Таблица 1.4 – Меры по предотвращению или уменьшению вероятности устранения рисков и снижения критичности последствий их возникновения

Риск	Меры
Ошибки в работе продукта, не полная или не качественная реализация функционала	Проведение ручного и функционального тестирования и последующее исправление ошибок
Утрата результатов деятельности	Обязательное сохранение прогресса разработки на нескольких платформах: github и gitflic
Неверное математическое обоснование	Обязательное использование ограничений на уровне СУБД и использование строгих типов данных
Компрометация учётных данных пользователей	Хранение паролей в базе данных только в виде хэша
Нестабильность и низкая производительность приложения	Невозможность запуска нескольких экземпляров приложений, невозможность одновременного проведения нескольких партий одновременно

Окончание таблицы 1.4

Риск	Меры
Использование программного продукта не по прямому назначению	Разработка руководства пользователя и технической документации и помещение их в директорию игры
Компрометация базы данных	Хранение паролей в базе данных только в виде хэша
НСД к системе	Использование принципа наименьших привилегий и использование role-based access control
Недоступность системы для легитимных пользователей	Регулярное проведение автоматических сохранений каждого действия в базе данных
Утрата игрового прогресса, подмена данных, неработоспособность приложения	Регулярное проведение автоматических сохранений каждого действия в базе данных
Нарушение игрового баланса, несоответствие ожидаемому поведению системы, сложность отладки	Проведение ручного и функционального тестирования и последующее исправление ошибок
Некорректная обработка специальных символов	Внедрение экранирования путём внедрения параметризованных запросов

Таким образом, в результате анализа были ли определены основные риски, вероятности их возникновения и критичность их возникновения, а также разработан комплекс технических и организационных мер, направленный на снижение данных показателей рисков.

2 РАЗРАБОТКА СТРУКТУРНОЙ, ФУНКЦИОНАЛЬНОЙ И ИНФОРМАЦИОННОЙ МОДЕЛЕЙ

2.1 Описание процессов системы в методологии функционального моделирования IDEF0

Была спроектирована функциональная модель схемы. На рисунке 2.1 представлен чёрный ящик в нотации IDEF0 [2]. Основной функцией данного процесса является реализация функций авторизации и регистрации и функций игровой логики. Процесс функционирования начинается с запуска приложения, продолжается процессом авторизации и регистрации пользователя при использовании логина и пароля. После успешной авторизации пользователь получает доступ к функциям игровой логики: начать игру, продолжить игру и загрузить игру. На рисунке 2.1 представлен чёрный ящик.

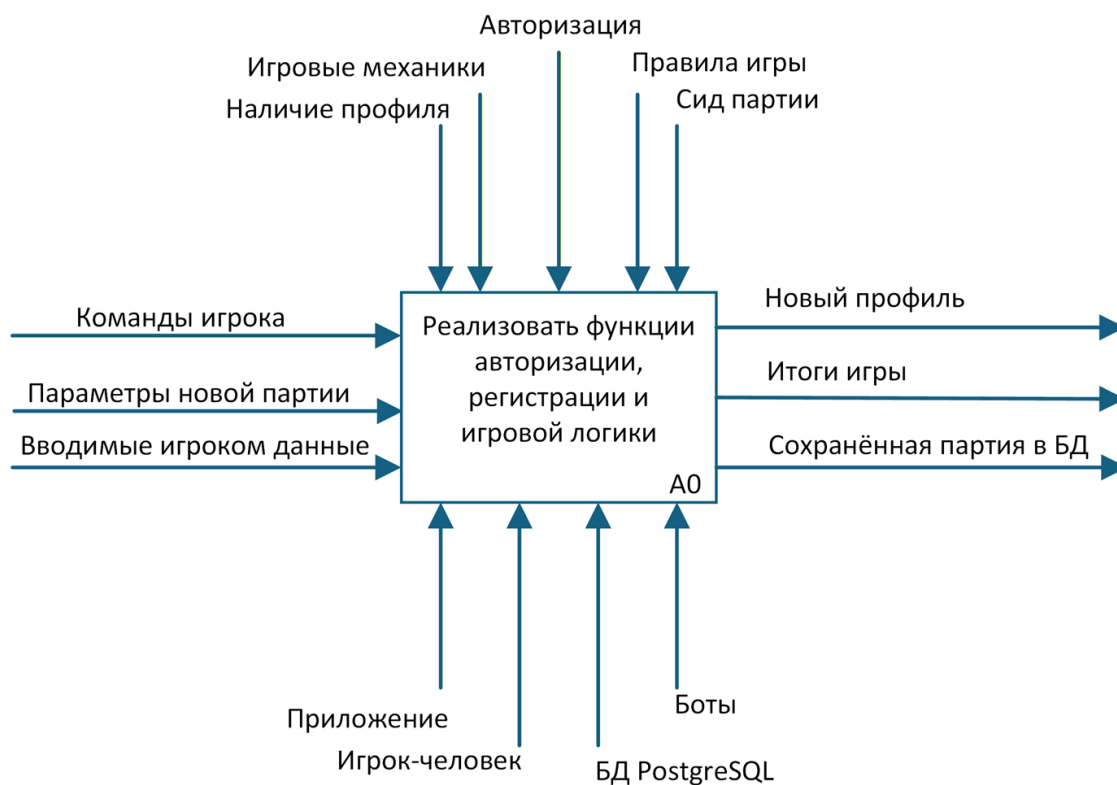


Рисунок 2.1 – Чёрный ящик

Для корректной работы системы не требуются определённые входные данные, так как пользователь при авторизации вводит данные, которые будут обработаны параметризованными запросами, возможные команды игрока жестко привязаны к функциям и выполняют строго определённые действия, а параметры партии указываются игроком в соответствии с определёнными правилами. Управляющими воздействиями выступают такие параметры, как наличие профиля, игровые механики, авторизация, правила игры и сид партии. Механизмами выполнения будут выступать само приложение, сам игрок, база данных и боты. Указанные механизмы обеспечивают выполнение функций, предусмотренных в системе.

Результатом работы системы является информация о завершённой или приостановленной партии и данные нового профиля.

Таким образом процесс «Реализовать функции авторизации, регистрации и игровой логики» представляет собой комплексную функцию, обеспечивающую автоматизацию основных процессов игры.

Для более детального представления работы приложения была выполнена декомпозиция контекстной диаграммы. Декомпозиция представлена в приложении Г.

Управлять профилями (A1). На вход функции подаются вводимые игроком данные и его команды. В процессе управления профилями осуществляется проверка введенных игроком данных по наличию авторизованного профиля в системе с такими данными и правилами авторизации.

Результатом выполнения функции является новый профиль или активная партия.

Управлять профилями (A2). На вход функции поступают параметры новой партии и команды игрока. На основе этих данных формируется игровая партия, сохраненная в бд или активная партия, в соответствии с правилами авторизации и правилами игры.

Выполнить вход (A3). На вход функции подаётся активная партия, команды игрока и обновленное состояние игры. Они обрабатываются в соответствии с правилами игры. Результатом функции будет Обновленное состояние партии

Управлять собственностью и инвентарём (A4). На вход функции подается обновлённое состояние партии, обрабатываемое в соответствии с игровыми механиками и сидом партии.

Далее у игрока появляется выбор действий: купить усиление, продать усиление, купить собственность, продать собственность, заплатить аренду и применить усиление.

Для более детального описания процесса выбора была выполнена декомпозиция A4, представленная в приложении Г.

Купить усиление (A41). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками и в строгом соответствии с сидом партии. Результатом функции будет обновленное состояние партии.

Продать усиление (A42). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками. Результатом функции будет обновленное состояние партии.

Купить собственность (A43). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками. Результатом функции будет обновленное состояние партии.

Продать собственность (A44). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками. Результатом функции будет обновленное состояние партии.

Заплатить аренду (A45). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками. Результатом функции будет обновленное состояние партии.

Применить усиление (A46). На вход функции подаётся изменённое состояние партии, обрабатываемое в соответствии с игровыми механиками. Результатом функции будет обновлённое состояние партии.

Закончить игру (A5). На вход функции подается измененное состояние партии, которое обрабатывается в соответствии с игровыми механиками и правилами игры. Результатом функции будет сохраненная в бд партия и итоги игры.

2.2 Информационная модель базы данных

В рамках работы была спроектирована модель базы данных в нотации ER [3]. Модель отображает основные объекты предметной области игры и взаимосвязи между ними. Нотация ER применяется для описания структуры данных, сущностей, ключей, ключей и связей, поэтому она была использована в качестве основы для последующего создания реляционной базы данных. Информационная модель представлена в приложении Г.

Диаграмма включает следующие сущности: «Игрок», «Игра», «Правила игры», «Участник игры», «Игровое поле», «Собственность», «Усиление», «Магазин усилений», «Слот магазина» и «Инвентарь».

Сущность «Игрок» связана с сущностью «Участник игры» отношением «Является» (1:M) – один игрок может участвовать в нескольких партиях. Содержит в себе следующие атрибуты:

1. ID Игрока(PK);
2. Имя;
3. Тип;
4. Хэш пароля.

Сущность «Игра» связана с «Участником игры» отношением «Содержит» (1:M) и с «Правилами игры» отношением «Следует» (1:1) – каждая игра имеет ровно один набор правил. Содержит в себе следующие атрибуты:

1. ID Игры(PK);
2. Статус;
3. Дата создания;
4. Дата последнего сохранения;
5. Сид.

Сущность «Игровое поле» связана с «Игрой» отношением «Содержит» (1:M) – одна игра содержит множество клеток поля. Содержит в себе следующие атрибуты:

1. ID Поля(PK);
2. ID Игры(FK);
3. Тип.

Сущность «Собственность» связана с «Игровым полем» отношением «Расположена на» (1:1) и с «Участником игры» отношением «Имеет» (1:M) – участник может владеть несколькими объектами собственности. Содержит в себе следующие атрибуты:

1. ID Собственности(PK);
2. ID Поля(FK);
3. ID Участника игры(FK);
4. Название;
5. Стоимость покупки;
6. Стоимость аренды;
7. Усиления.

Сущность «Магазин усилений» также связана с «Игровым полем» отношением «Расположена на» (1:1) и состоит из слотов посредством связи «Состоит из» (1:M). Содержит в себе следующие атрибуты:

1. ID Магазина(PK);
2. ID Поля (FK);
3. ID Игры(FK);
4. Смещение.

Сущность «Усиление» связана со «Слотом магазина» отношением «Содержится в» (1:M) и с «Инвентарём» через отношение «Имеет» (1:M).

Содержит в себе следующие атрибуты:

1. ID Усиления(PK);
2. Название;
3. Описание;
4. Стоимость;
5. Эффект.

Сущность «Инвентарь» связана с «Участником игры» отношением «Имеет» (1:M). Содержит в себе следующие атрибуты:

1. ID Участника игры(PK);
2. ID Усиления(FK).

Сущность «Участник игры» содержит в себе следующие атрибуты:

1. ID Игры(PK);
2. ID Игрока(PK);
3. Текущая позиция;
4. Баланс;
5. Сделано ходов;
6. Потрачено;
7. Получено.

Таким образом разработанная информационная модель охватывает все основные сущности, атрибуты и связи предметной области. Использование связей между таблицами обеспечивает целостность данных и позволяет автоматизировать игровые процессы.

2.3 Структурная модель системы

Для описания структуры автоматизированной системы «DreamBreaker» была разработана структурная модель в виде UML диаграммы классов [4]. Данная модель отображает основные сущности предметной области, их атрибуты и методы их взаимодействия. В процессе разработки заданную структуру пришлось модифицировать. Структурная модель представлена в приложении Г.

Диаграмма включает следующие ключевые компоненты системы:

1. Struct Player представляет сущность игрока и содержит идентификатор, имя, тип и хеш пароля. Struct Participant представляет участника конкретной игровой партии и хранит идентификаторы игрока и игры, текущую позицию на поле, баланс, количество совершённых ходов, а также суммарные показатели потраченных и заработанных средств. Struct Game содержит данные об игровой партии: идентификатор, статус, дату создания, дату последнего сохранения, зерно генератора и правила игры. Struct GameRules инкапсулирован внутри Game и определяет стартовый баланс, максимальное число ходов и целевой баланс;

2. Struct Property представляет объект собственности на игровом поле и содержит идентификатор связанной клетки, идентификатор владельца, название, стоимость покупки и аренды, а также метод calculate_rent для вычисления итоговой суммы аренды с учётом усиления. Struct PowerUp описывает усиление с его названием, описанием, стоимостью и эффектом;

3. Struct GameEngine реализует игровой движок и агрегирует объект Game и список участников. Он предоставляет методы make_turn для выполнения хода, next_turn для передачи хода следующему участнику и check_end_condition для проверки условий завершения партии;

4. Trait Repository<T> определяет обобщённый интерфейс доступа к данным с методами find_by_id, save, delete и find_all. Structs PlayerRepository и GameRepository реализуют данный trait для сущностей игрока и игры

соответственно и используют DbPool для подключения к базе данных PostgreSQL через пул соединений sqlx::PgPool;

5. Trait CellHandler определяет интерфейс обработки игровых событий при попадании участника на клетку поля. Enum CellType перечисляет возможные типы клеток: Empty, Property, Shop, Start и Tax. Struct ShopGenerator отвечает за генерацию ассортимента магазина усилений на основе зерна генератора;

6. В процессе реализации приложения исходная проектная структура была адаптирована: паттерн Repository заменён прямыми вызовами хранимых функций PostgreSQL через модуль db.rs, а логика игрового движка вынесена на уровень базы данных. Отрисовка интерфейса реализована средствами фреймворка Iced на основе реактивной модели сообщений, что не предполагает выделения отдельных классов-обработчиков.

2.4 Информационная модель системы

Для описания информационных потоков автоматизированной системы была разработана диаграмма потоков данных [5]. Данная диаграмма позволяет отразить внешние сущности, процессы, хранилища данных и направления передачи информации между ними.

На диаграмме выделены основные внешние сущности: игрок-человек и боты. Игрок-человек взаимодействует с системой при аутентификации, управлении игрой, выполнении игровых действий и совершении покупок в магазине усилений. Боты участвуют в игровом процессе в качестве автоматических противников, выполняя ходы на основе данных игрового поля.

Основными процессами системы являются:

1. Аутентификация. В рамках данного процесса игрок передаёт логин и пароль, система обращается к хранилищу пользователей и игр для получения хэша пароля и проверки учётных данных. По результатам проверки игроку возвращается результат входа;

2. Управление игрой. Данный процесс обеспечивает создание и загрузку игровых сессий. Игрок передаёт запрос на создание или загрузку игры, система сохраняет и читает данные из хранилища пользователей и игр, после чего передаёт статус игры и инициирует старт хода игрока или бота;

3. Ход игрока. В рамках данного процесса обрабатывается бросок кубика и игровые действия игрока. Система читает и обновляет данные клеток из хранилища игрового поля, а также запрашивает эффекты усилений из хранилища инвентаря и усилений. Результаты хода в виде нового состояния передаются игроку;

4. Ход бота. Данный процесс аналогичен ходу игрока и выполняется автоматически. Боты передают позицию и результат броска кубика, система обновляет игровое поле и возвращает результат хода обратно в процесс управления игрой;

5. Магазин усилений. При попадании игрока на клетку магазина инициируется данный процесс. Игрок может приобрести усиление, после чего система обновляет инвентарь в хранилище усилений и возвращает игроку подтверждение покупки;

В функциональной модели используются следующие хранилища данных:

- D1 – пользователи и игры. Хранилище содержит данные об учётных записях пользователей, созданных игровых сессиях и их состоянии;
- D2 – игровое поле. Хранилище содержит данные о клетках поля, их типах, собственниках и текущем состоянии в рамках игровой сессии;
- D3 – инвентарь и усиления. Хранилище содержит данные об усилении игрока, слотах магазина и эффектах, применённых к собственности.

Таким образом, разработанная информационная модель описывает основные игровые процессы, потоки данных и взаимодействие участников с системой, что позволяет определить состав реализуемых функций и структуру дальнейшей разработки программного продукта. Диаграмма потоков данных представлена в приложении Г.

3 РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ

3.1 Создание таблиц

Реализация базы данных выполнялась на основе информационной модели, разработанной в нотации ER в разделе 2. Логическая модель была преобразована в физическую модель реляционной базы данных PostgreSQL. Использование PostgreSQL позволило реализовать таблицы, связи между ними, ограничения целостности, индексы и триггеры на уровне базы данных [6]. Для каждой сущности предметной области была создана отдельная таблица. В таблицах 3.1–3.9 приведены описания таблиц базы данных, включая атрибуты, типы данных и ограничения целостности. Дополнительно на уровне базы данных реализованы триггеры и хранимые функции, обеспечивающие контроль бизнес-правил системы бронирования и продажи билетов.

Таблица 3.1 – Глоссарий сущности «users»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически (gen_random_uuid())
username	VARCHAR(50)	Обязательное поле, уникальное, длина от 1 до 50 символов
type	VARCHAR(20)	Обязательное поле, значение по умолчанию: human; допустимые значения: human, bot, admin
password_hash	VARCHAR(100)	Обязательное поле, хэш пароля (bcrypt)
created_at	TIMESTAMPTZ	Обязательное поле, значение по умолчанию: NOW()
is_active	BOOLEAN	Обязательное поле, значение по умолчанию: ИСТИНА

Таблица 3.2 – Глоссарий сущности «games»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
status	VARCHAR(20)	Обязательное поле, значение по умолчанию: pending; допустимые значения: pending, active, paused, finished, surrender

Окончание таблицы 3.2

Атрибут	Тип данных	Ограничения
seed	BIGINT	Обязательное поле, сид генератора случайных чисел
created_at	TIMESTAMPTZ	Обязательное поле, значение по умолчанию: NOW()
last_saved_at	TIMESTAMPTZ	Обязательное поле, значение по умолчанию: NOW(); обновляется триггером при каждом UPDATE

Таблица 3.3 – Глоссарий сущности «game_rules»

Атрибут	Тип данных	Ограничения
game_id	UUID	Первичный ключ и внешний ключ → games(id), каскадное удаление
starting_balance	BIGINT	Обязательное поле, значение по умолчанию: 1000; начальный баланс участников
max_turns	INT	Обязательное поле, значение по умолчанию: 50; максимальное число ходов
target_balance	BIGINT	Обязательное поле, значение по умолчанию: 2500; целевой баланс для победы

Таблица 3.4 – Глоссарий сущности «game_participants»

Атрибут	Тип данных	Ограничения
game_id	UUID	Часть составного первичного ключа, внешний ключ → games(id), каскадное удаление
user_id	UUID	Часть составного первичного ключа, внешний ключ → users(id), каскадное удаление
position	INT	Обязательное поле, значение по умолчанию: 0; текущая позиция на поле (0–39)
balance	BIGINT	Обязательное поле, значение по умолчанию: 1000; текущий баланс участника
moves_made	INT	Обязательное поле, значение по умолчанию: 0; количество совершённых ходов
total_spent	BIGINT	Обязательное поле, значение по умолчанию: 0; суммарно потрачено за игру
total_earned	BIGINT	Обязательное поле, значение по умолчанию: 0; суммарно заработано за игру
turn_order	INT	Обязательное поле, порядок хода участника в рамках игры

Таблица 3.5 – Глоссарий сущности «game_cells»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
game_id	UUID	Обязательное поле, внешний ключ → games(id), каскадное удаление
cell_index	INT	Обязательное поле, индекс клетки на поле (0–39); уникален в рамках игры совместно с game_id
cell_type	VARCHAR(20)	Обязательное поле; допустимые значения: empty, property, shop, start, tax
tax_amount	BIGINT	Необязательное поле, значение по умолчанию: 0; сумма налога для клетки типа tax
property_id	UUID	Необязательное поле, внешний ключ → properties(id), SET NULL при удалении
shop_id	UUID	Необязательное поле, внешний ключ → shops(id), SET NULL при удалении

Таблица 3.6 – Глоссарий сущности «properties»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
cell_id	UUID	Необязательное поле, внешний ключ → game_cells(id), каскадное удаление
owner_game_id	UUID	Необязательное поле, часть внешнего ключа → game_participants(game_id, user_id); NULL означает, что собственность не куплена
owner_user_id	UUID	Необязательное поле, часть внешнего ключа → game_participants(user_id); NULL означает, что собственность не куплена
name	VARCHAR(100)	Обязательное поле, название собственности
purchase_cost	BIGINT	Обязательное поле, стоимость покупки
rent_cost	BIGINT	Обязательное поле, базовая стоимость аренды
upgrades	JSONB	Необязательное поле, значение по умолчанию: []; массив установленных усилений
created_at	TIMESTAMPTZ	Обязательное поле, значение по умолчанию: NOW()

Таблица 3.7 – Глоссарий сущности «power_ups»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
name	VARCHAR(100)	Обязательное поле, название усиления

Окончание таблицы 3.7

Атрибут	Тип данных	Ограничения
description	TEXT	Необязательное поле, описание усиления
cost	BIGINT	Обязательное поле, стоимость усиления в магазине
effect	JSONB	Обязательное поле, JSON-объект эффекта; допустимые типы: flat_base, percent_bonus, flat_final

Таблица 3.8 – Глоссарий сущности «shops»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
cell_id	UUID	Необязательное поле, внешний ключ → game_cells(id), каскадное удаление
game_id	UUID	Обязательное поле, внешний ключ → games(id), каскадное удаление
offset_value	INT	Обязательное поле, значение по умолчанию: 0; смещение генерации ассортимента

Таблица 3.9 – Глоссарий сущности «shop_slots»

Атрибут	Тип данных	Ограничения
id	UUID	Первичный ключ, генерируется автоматически
shop_id	UUID	Обязательное поле, внешний ключ → shops(id), каскадное удаление
power_up_id	UUID	Необязательное поле, внешний ключ → power_ups(id), SET NULL при удалении
slot_index	INT	Обязательное поле, номер слота; уникален совместно с shop_id
status	VARCHAR(20)	Обязательное поле, значение по умолчанию: available; допустимые значения: available, sold, locked
cost	BIGINT	Обязательное поле, стоимость усиления в данном слоте

Таблица 3.10 – Глоссарий сущности «player_inventory»

Атрибут	Тип данных	Ограничения
game_id	UUID	Часть составного первичного ключа, внешний ключ (совместно с user_id) → game_participants, каскадное удаление
user_id	UUID	Часть составного первичного ключа, внешний ключ → users(id), каскадное удаление

Окончание таблицы 3.10

Атрибут	Тип данных	Ограничения
power_up_id	UUID	Часть составного первичного ключа, внешний ключ → power_ups(id), каскадное удаление
quantity	INT	Обязательное поле, значение по умолчанию: 1; количество единиц усиления в инвентаре

На основании приведённого глоссария сущностей были разработаны SQL-скрипты, которые последовательно создают все таблицы базы данных, устанавливают связи между ними посредством внешних ключей и задают правила поведения при удалении записей. Все скрипты создания базы данных расположены в директории migrations в корне проекта. Ссылки на репозитории приведены в приложении Д.

3.2 Реализация триггеров и хранимых функций

Для обеспечения целостности данных и автоматизации игровой логики в базе данных реализованы триггеры и хранимые функции. Их использование позволяет исключить выполнение ряда проверок на стороне клиентского приложения и гарантировать соблюдение правил предметной области независимо от способа обращения к базе данных.

В системе реализованы следующие основные механизмы:

- проверка активности игры перед выполнением любой операции, изменяющей её состояние;
- регистрация нового пользователя с сохранением предварительно хэшированного пароля;
- автоматическое начисление бонуса за прохождение или попадание на стартовую клетку при фиксации хода участника;
- покупка собственности с проверкой достаточности баланса и присвоением владельца;

- списание арендной платы с арендатора и зачисление её владельцу собственности;
- расчёт итоговой стоимости аренды с учётом установленных усилений;
- начисление налога в размере $100 + 5\%$ от текущего баланса участника;
- покупка усиления из магазина с проверкой баланса и обновлением инвентаря;
- реролл ассортимента магазина с псевдослучайным перемешиванием на основе сйда игры;
- продажа усиления из инвентаря по цене 50% от стоимости покупки;
- установка и снятие усиления в слот собственности с перемещением между инвентарём и объектом;
- выполнение полного хода бота: перемещение, покупка свободной собственности, оплата аренды или налога, объявление банкротства при нехватке средств;
- сохранение игры с переводом в статус `paused` и сдача с переводом в статус `surrender`;
- инициализация игрового поля из 40 клеток с генерацией собственности и заполнением магазинов при создании новой партии;
- автоматическое обновление временной метки последнего сохранения игры при каждом изменении записи в таблице `games`.

Использование хранимых функций с атрибутом `SECURITY DEFINER` позволило перенести критически важную бизнес-логику на уровень базы данных, что повышает надёжность системы и обеспечивает единообразную обработку данных независимо от клиентского приложения. Скрипты создания триггеров и хранимых функций расположены в директории `migrations` в корне проекта. Ссылки на репозитории приведены в приложении Д.

4 РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ

4.1 Архитектура серверной части

Серверная часть системы реализована в виде базы данных PostgreSQL 18, которая выступает не просто хранилищем данных, но и основным исполнителем бизнес-логики приложения. Вся обработка игровых операций инкапсулирована в хранимых функциях, написанных на языке PL/pgSQL. Клиентское приложение не выполняет прямых SQL-запросов к таблицам – любое взаимодействие с данными осуществляется исключительно через вызов хранимых функций.

Архитектура серверной части построена по функциональному принципу и включает несколько взаимосвязанных уровней.

Таблицы базы данных хранят состояние всех сущностей системы: пользователей, игр, участников, игрового поля, собственности, магазинов и инвентаря. Прямой доступ к таблицам из приложения закрыт – они являются внутренним уровнем хранения данных.

Хранимые функции принимают параметры от клиента, выполняют проверку корректности входных данных, реализуют бизнес-правила предметной области, изменяют состояние таблиц и возвращают результат выполнения операции. Для каждой функциональной подсистемы реализован отдельный набор функций: управление пользователями, создание и ведение игр, операции игрового процесса, магазин усилений, управление собственностью и статистика.

Триггеры выполняют дополнительные проверки и автоматические действия при изменении данных без участия клиентского приложения. В системе реализован триггер автоматического обновления временной метки последнего сохранения игры при каждом изменении записи в таблице games.

Функции, объявленные с атрибутом SECURITY DEFINER, выполняются с правами владельца функции, что обеспечивает единообразный контроль

доступа к данным независимо от того, каким способом выполняется обращение к базе данных.

Взаимодействие компонентов осуществляется последовательно. Клиентское приложение формирует вызов хранимой функции с необходимыми параметрами, функция выполняет проверки и операции над данными, после чего возвращает результат приложению. При необходимости внутри одной функции последовательно вызываются другие хранимые функции – например, `create_game_full` внутри себя вызывает `init_game_board`, которая инициализирует всё игровое поле в рамках одной транзакции.

Такая организация серверной части позволяет сосредоточить всю критически важную логику на уровне базы данных, упрощает сопровождение системы и обеспечивает целостность данных независимо от клиентского приложения.

4.2 Аутентификация пользователей

Для обеспечения безопасности системы реализован механизм аутентификации на основе хэширования паролей с использованием алгоритма `bcrypt`. Поскольку приложение является десктопным и работает без промежуточного сервера, хранение сессий и токенов доступа не требуется – проверка подлинности выполняется однократно при входе в систему.

Процесс аутентификации начинается с ввода пользователем имени и пароля. Приложение вызывает хранимую функцию `get_user_credentials`, которая выполняет поиск пользователя в таблице `users` базы данных PostgreSQL и возвращает хэш пароля и статус учётной записи. Для защиты учётных данных в базе хранятся не сами пароли, а их хэшированные значения, сформированные с использованием алгоритма `bcrypt`.

Проверка введённого пароля выполняется на стороне приложения: функция `bcrypt::verify` сравнивает введённое значение с хранимым хэшем. Если учётная запись заблокирована (`is_active = false`) или пароль не совпадает,

пользователю возвращается сообщение об ошибке без раскрытия причины несоответствия. После успешной проверки приложение получает полные данные пользователя через функцию `get_user_by_id` и сохраняет идентификатор и имя пользователя в состоянии приложения. Эти данные используются во всех последующих обращениях к базе данных в рамках текущего сеанса работы.

При регистрации нового пользователя хэш пароля вычисляется в приложении функцией `bcrypt::hash` с параметром `DEFAULT_COST` до передачи данных в базу. Таким образом, открытый пароль никогда не передаётся в базу данных и не сохраняется ни в каком виде, кроме хэша.

В системе предусмотрены два типа учётных записей: `human` для игроков и `bot` для управляемых базой данных участников. Учётные записи ботов создаются при инициализации базы данных и недоступны для входа через форму аутентификации.

4.3 Работа с данными и бизнес-логика

Доступ к данным в системе осуществляется исключительно через вызовы хранимых функций PostgreSQL. Для взаимодействия с базой данных в модуле `db.rs` используется единый пул соединений на основе библиотеки `sqlx`, который управляет подключениями и обеспечивает выполнение асинхронных запросов из всех компонентов приложения.

Каждая операция над данными оформлена в виде отдельной асинхронной функции в модуле `db.rs`. Функция формирует вызов хранимой процедуры PostgreSQL с параметрами, передаёт его в базу данных и десериализует результат в типизированную структуру Rust. Для передачи данных используются исключительно параметризованные выражения вида `$1`, `$2`, ..., что полностью исключает возможность SQL-инъекций. На рисунке 3.1 приведён пример вызова хранимой функции из кода приложения.

```
pub async fn buy_property(
    pool: &PgPool,
    game_id: Uuid,
    user_id: Uuid,
    cell_index: i32,
) -> Result<ParticipantState, DbError> {
    let state: ParticipantState = sqlx::query_as::<_ = Postgres, ParticipantState>(
        sql: "SELECT \"position\", balance, moves_made, total_spent, total_earned
        | FROM buy_property($1, $2, $3)",
    )
    .bind(game_id)
    .bind(user_id)
    .bind(cell_index)
    .fetch_one(executor: pool)
    .await?;
    Ok(state)
}
```

Рисунок 3.1 – Пример вызова хранимой функции

Бизнес-логика системы сосредоточена преимущественно на стороне базы данных. Клиентское приложение отвечает за отображение интерфейса, обработку пользовательского ввода, бросок кубиков и управление очередностью ходов. Все операции, изменяющие состояние игры, делегируются хранимым функциям PostgreSQL: проверка допустимости операции, изменение данных и возврат обновлённого состояния выполняются на стороне базы данных

Для обеспечения целостности данных в базе реализованы ограничения, триггеры и хранимые функции. Хранимые функции выполняют проверку активности игры перед любой операцией, контроль достаточности баланса при покупке собственности и оплате аренды, расчёт стоимости аренды с учётом установленных усилений, автоматическое начисление бонуса за прохождение стартовой клетки, обработку банкротства бота с освобождением его собственности. Триггер `trg_auto_save_game` автоматически обновляет временную метку последнего сохранения при каждом изменении записи игры.

Таким образом, контроль корректности данных обеспечивается непосредственно на уровне базы данных. Клиентское приложение не содержит дублирующих проверок игровых правил – это повышает надёжность системы

и гарантирует корректность данных независимо от способа обращения к базе. Полный исходный код серверной части расположен в директории migrations в корне проекта. Ссылки на репозитории приведены в приложении Д.

5 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ

5.1 Внутреннее устройство клиентского приложения

Клиентская часть системы DreamBreaker реализована на языке Rust [7] с использованием GUI-фреймворка Iced 0.13 [8]. Приложение является десктопным и не требует веб-браузера или промежуточного сервера – оно запускается напрямую на устройстве пользователя и взаимодействует с базой данных PostgreSQL через пул соединений библиотеки sqlx.

Основной точкой входа является функция `main` в файле `main.rs`. В ней считывается строка подключения к базе данных из файла `.env` посредством библиотеки `dotenvy`, после чего запускается приложение Iced с передачей трёх обязательных компонентов: функции обновления состояния `update`, функции построения интерфейса `view` и функции подписки на системные события `subscription`.

Исходный код приложения разделён на два модуля. Модуль `db.rs` отвечает исключительно за взаимодействие с базой данных: подключение, применение миграций и вызов хранимых функций PostgreSQL. Модуль `main.rs` содержит всё, что связано с пользовательским интерфейсом: определение состояния приложения, обработку событий и построение экранов.

Архитектура приложения построена по паттерну The Elm Architecture. Центральным элементом является структура DreamBreaker, хранящая полное состояние приложения. Все изменения состояния происходят исключительно через функцию `update`, которая принимает типизированное сообщение типа `Message` и возвращает новое состояние вместе с командой для выполнения асинхронной операции с базой данных. Функция `view` строит интерфейс на основе текущего состояния и не содержит никакой логики изменения данных.

Навигация между экранами реализована через перечисление `Screen` со следующими значениями: `Menu` – главное меню, `Login` – вход в систему, `Register` – регистрация, `LoadGame` – загрузка сохранённой игры, `CreateGame` –

создание новой партии, Game – игровой экран, GameOver – экран завершения игры, Settings – настройки приложения. Переход между экранами выполняется заменой поля screen в структуре состояния без какой-либо маршрутизации URL.

Состояние приложения содержит следующие группы полей: данные авторизованного пользователя (current_user, users), параметры активной игры (active_game_id, game_rules, player_state), данные игрового поля (board_cells, inventory, player_properties), параметры хода (turn_phase, dice_roll, cell_action), очередь ходов ботов (bot_turn_queue, bot_turn_log), данные магазина (shop_slots), параметры отображения (board_zoom, window_mode, window_width, window_height) и журналы событий (income_log, rent_received_log).

Таким образом, внутренняя структура клиентского приложения построена по принципу единого источника истины: всё состояние хранится в одной структуре, все изменения проходят через единую функцию обработки сообщений, а интерфейс всегда строится заново на основе актуального состояния. Такой подход упрощает отладку и исключает рассинхронизацию данных между экранами. Полный исходный код клиентской части расположен в директории src в корне проекта. Ссылки на репозитории приведены в приложении Д.

5.2 Интерфейс системы

Пользовательский интерфейс системы DreamBreaker реализован в едином тёмном стиле с использованием GUI-фреймворка Iced 0.13. Все элементы интерфейса имеют русскоязычные подписи. Масштабирование интерфейса выполняется автоматически в зависимости от размера окна.

На рисунке 5.1 представлен экран выбора профиля.

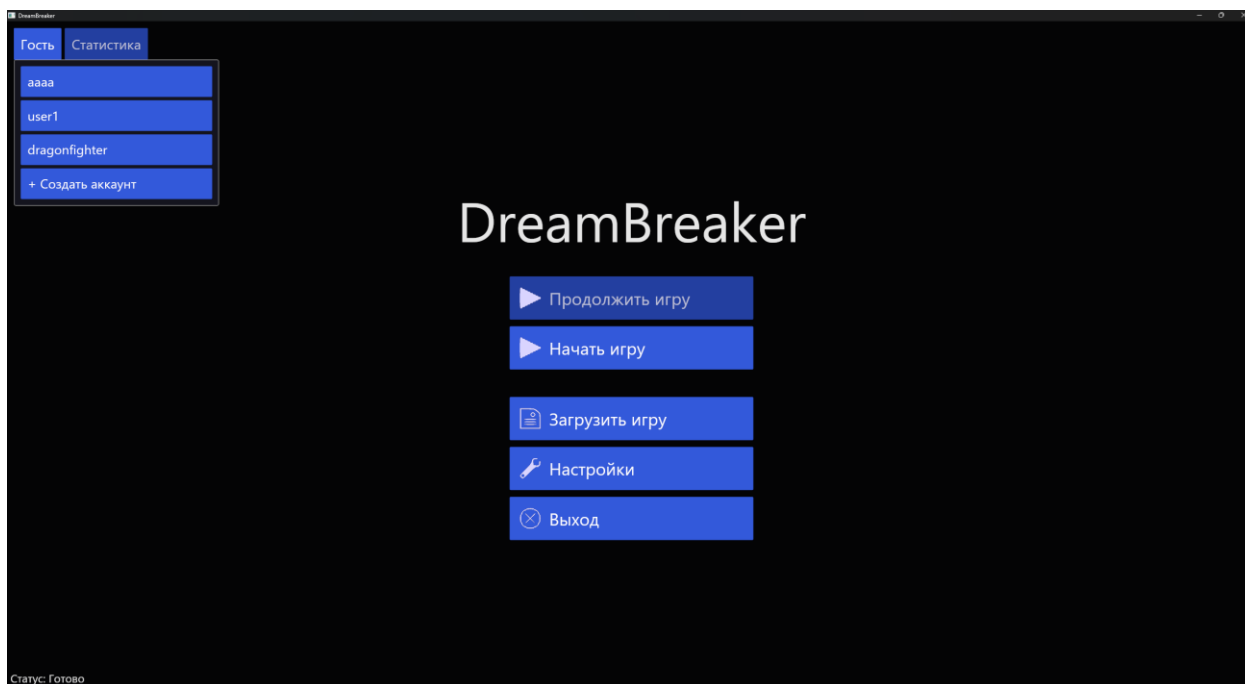


Рисунок 5.1 – Экран выбора профиля

На экране главного меню пользователь выбирает существующий профиль из списка или переходит к регистрации нового. Если у выбранного пользователя есть незавершённая партия, кнопка меняется на «Продолжить игру».

На рисунке 5.2 представлен экран входа в систему.

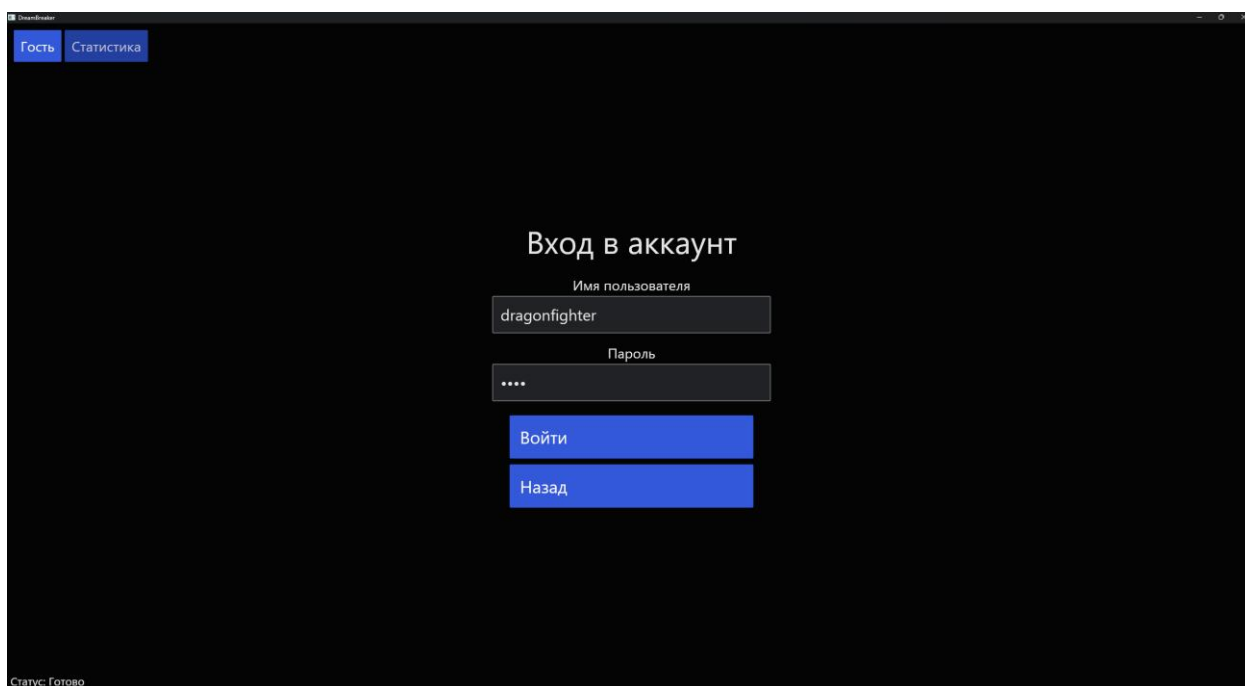


Рисунок 5.2 – Экран входа в систему

На экране авторизации пользователь вводит имя и пароль. Проверка пароля выполняется на стороне приложения с использованием алгоритма bcrypt. При вводе неверного пароля или отсутствии пользователя в базе данных выводится сообщение об ошибке.

На рисунке 5.3 представлен экран регистрации.

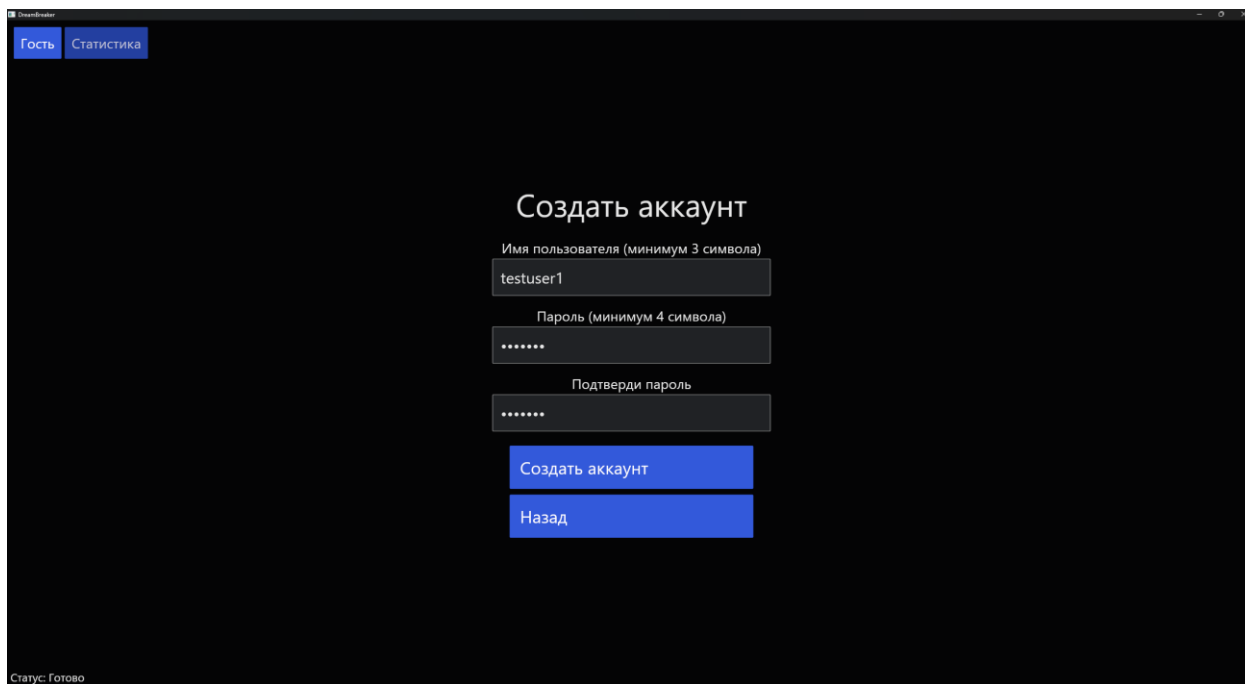


Рисунок 5.3 – Экран регистрации

На экране регистрации пользователь вводит имя и пароль. Выполняется проверка совпадения пароля и его подтверждения. При успешной регистрации пользователь автоматически переходит в главное меню.

На рисунке 5.4 представлен экран создания новой партии.

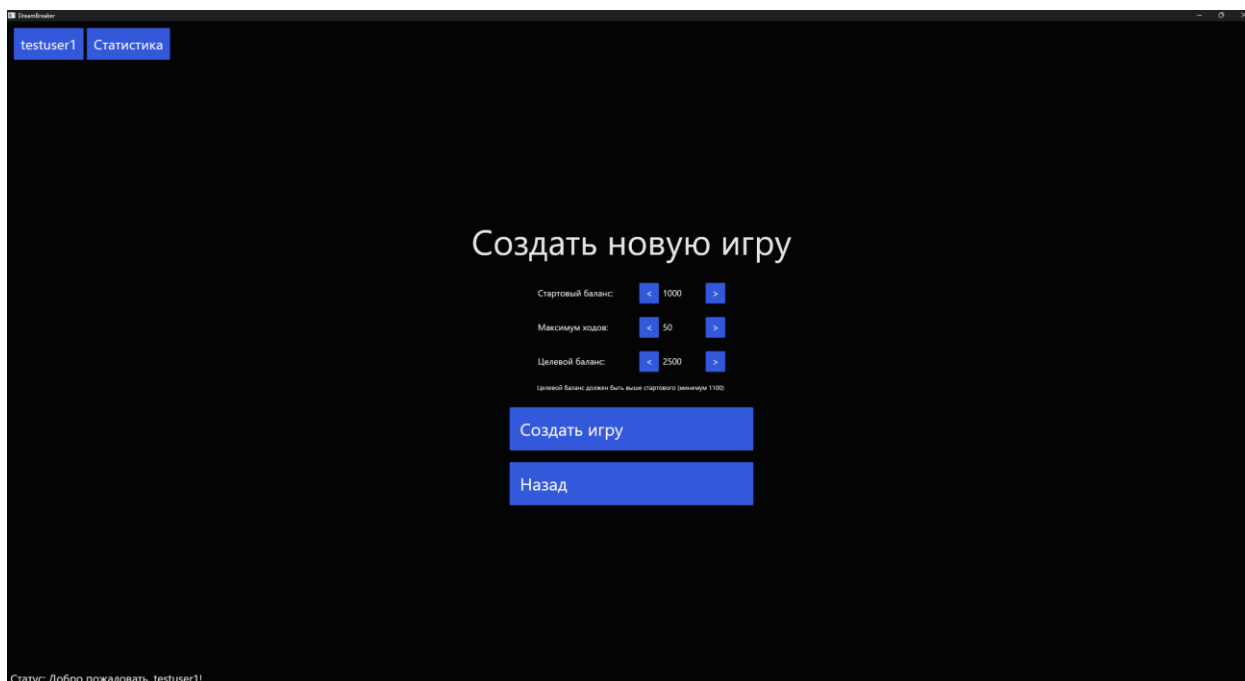


Рисунок 5.4 – Экран создания новой партии

Перед началом игры пользователь настраивает параметры партии: стартовый баланс, максимальное количество ходов и целевой баланс для победы. Значения изменяются кнопками увеличения и уменьшения.

На рисунке 5.5 представлен экран загрузки сохранённой игры.

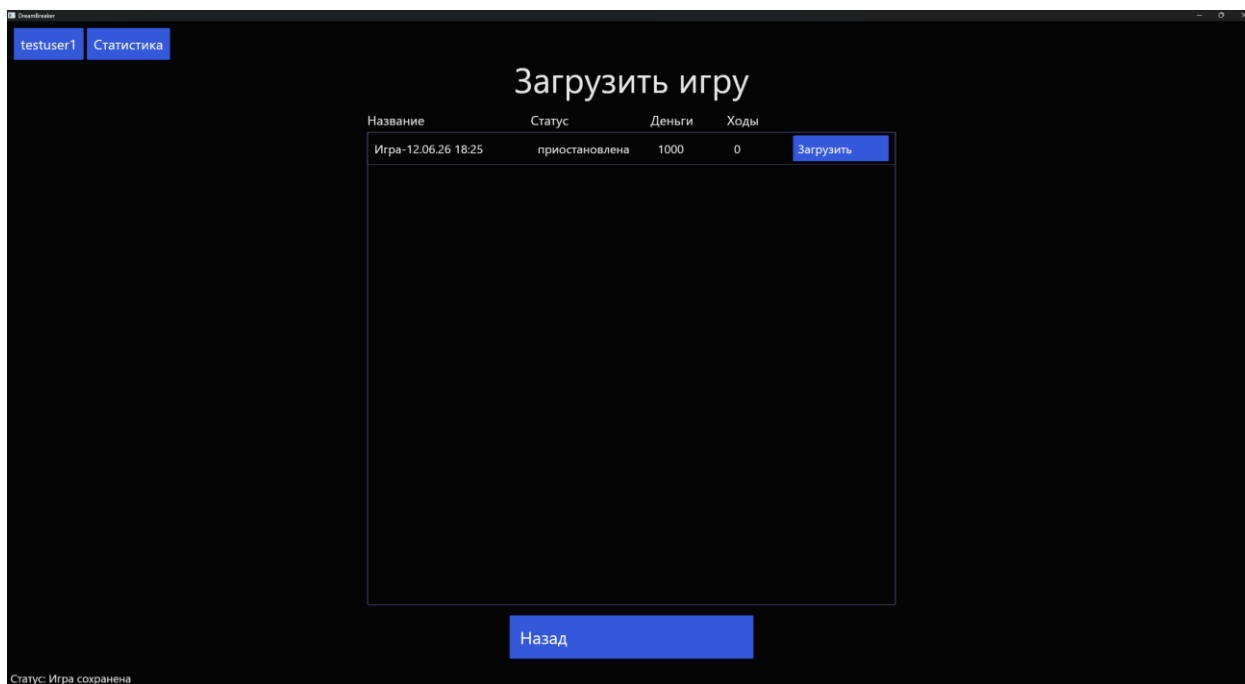


Рисунок 5.5 – Экран загрузки сохранённой игры

На экране загрузки отображается список всех партий пользователя с указанием статуса, текущего баланса и количества совершённых ходов. Пользователь выбирает нужную партию и продолжает игру.

На рисунке 5.6 представлен основной игровой экран.



Рисунок 5.6 – Основной игровой экран

Основной игровой экран включает игровое поле из 40 клеток, расположенных по периметру, правую информационную панель с текущим балансом, количеством ходов и инвентарём усилений, а также панель статуса ботов. Бросок кубиков выполняется кнопкой «Бросить кубики», после чего фишка игрока перемещается на новую клетку и определяется действие.

На рисунке 5.7 представлен лог ходов ботов.



Рисунок 5.7 – Лог ходов ботов

После завершения хода игрока автоматически выполняются ходы всех активных ботов. Каждое действие бота фиксируется в журнале событий: перемещение, покупка собственности, оплата аренды, уплата налога или банкротство.

На рисунках 5.8-5.10 представлены всплывающие окна действий на клетке.

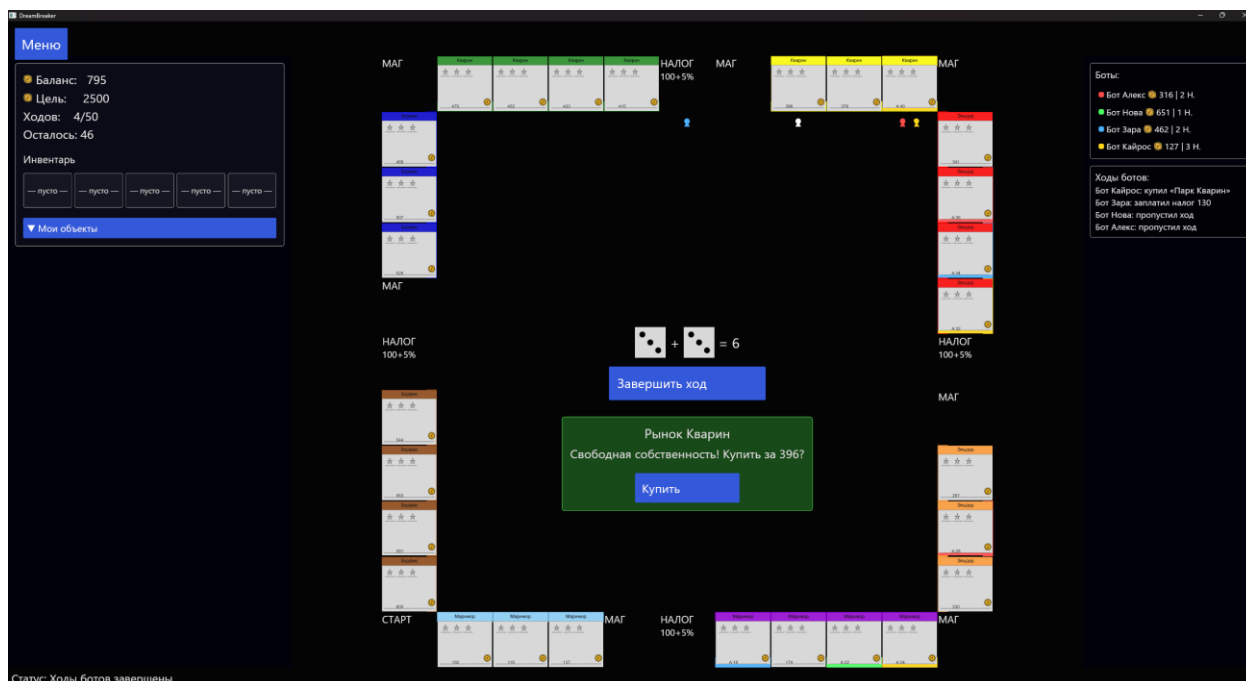


Рисунок 5.8 – Всплывающее окно действия на клетке «покупка»



Рисунок 5.9 – Всплывающее окно действия на клетке «аренда»

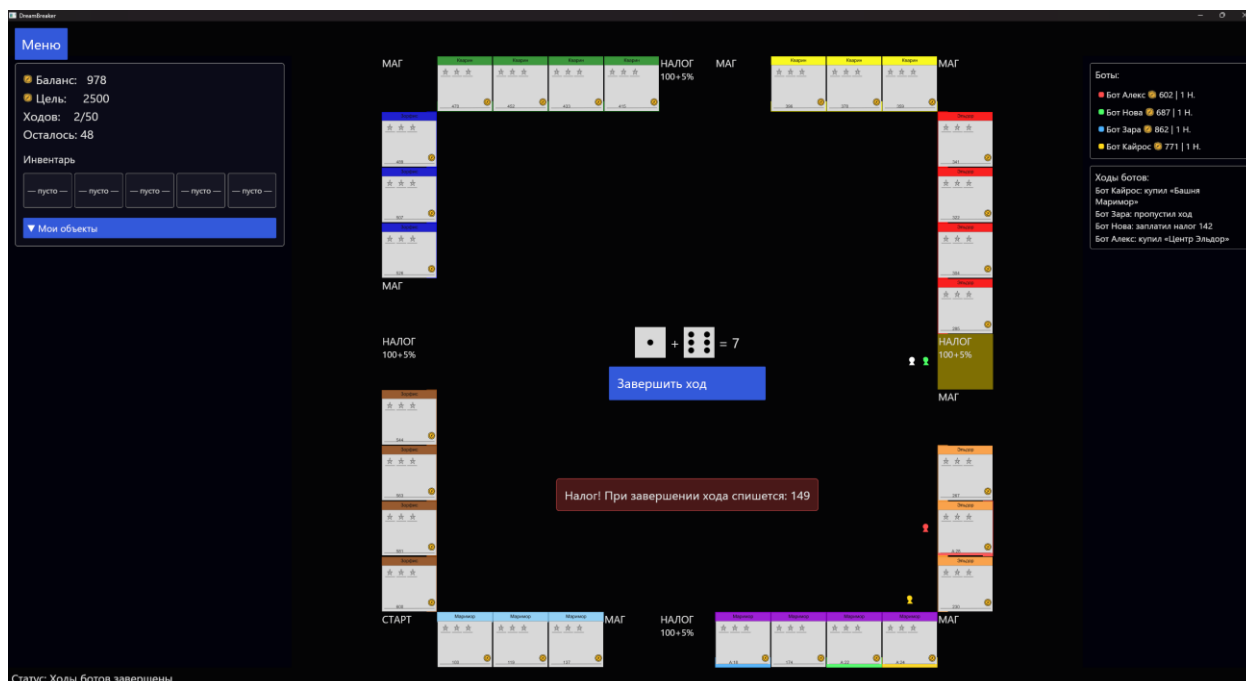


Рисунок 5.10 – Всплывающее окно действия на клетке «налог»

При попадании на клетку-собственность определяется её статус. Если собственность свободна, предлагается её купить. Если собственность принадлежит другому участнику, автоматически списывается арендная плата с учётом установленных усилений. При попадании на клетку налога списывается налог в размере фиксированной платы 100 + 5% от текущего баланса.

На рисунке 5.11 представлен экран магазина усилений.

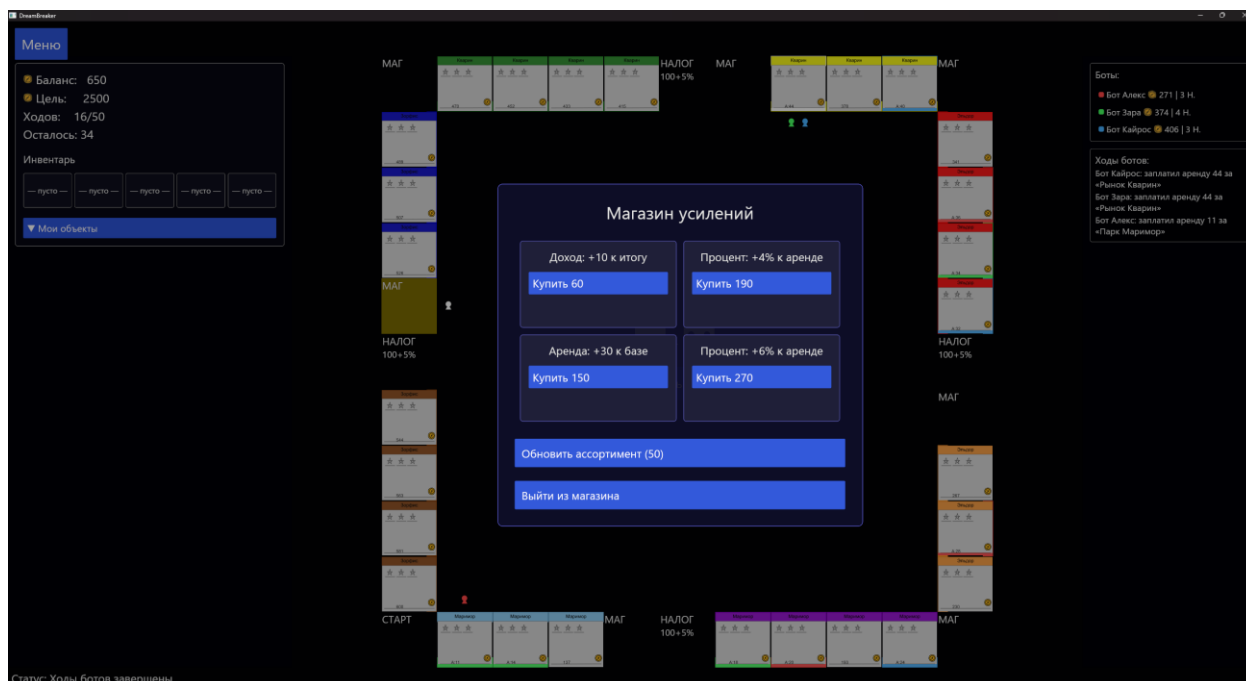


Рисунок 5.11 – Экран магазина усилений

Магазин усилений открывается при попадании на клетку-магазин. В нём представлены четыре слота с доступными усилениями. Каждое усиление имеет название, описание эффекта и стоимость. Ассортимент можно обновить за внутриигровой баланс— стоимость каждого последующего обновления увеличивается.

На рисунке 5.12 представлена всплывающая подсказка усиления.

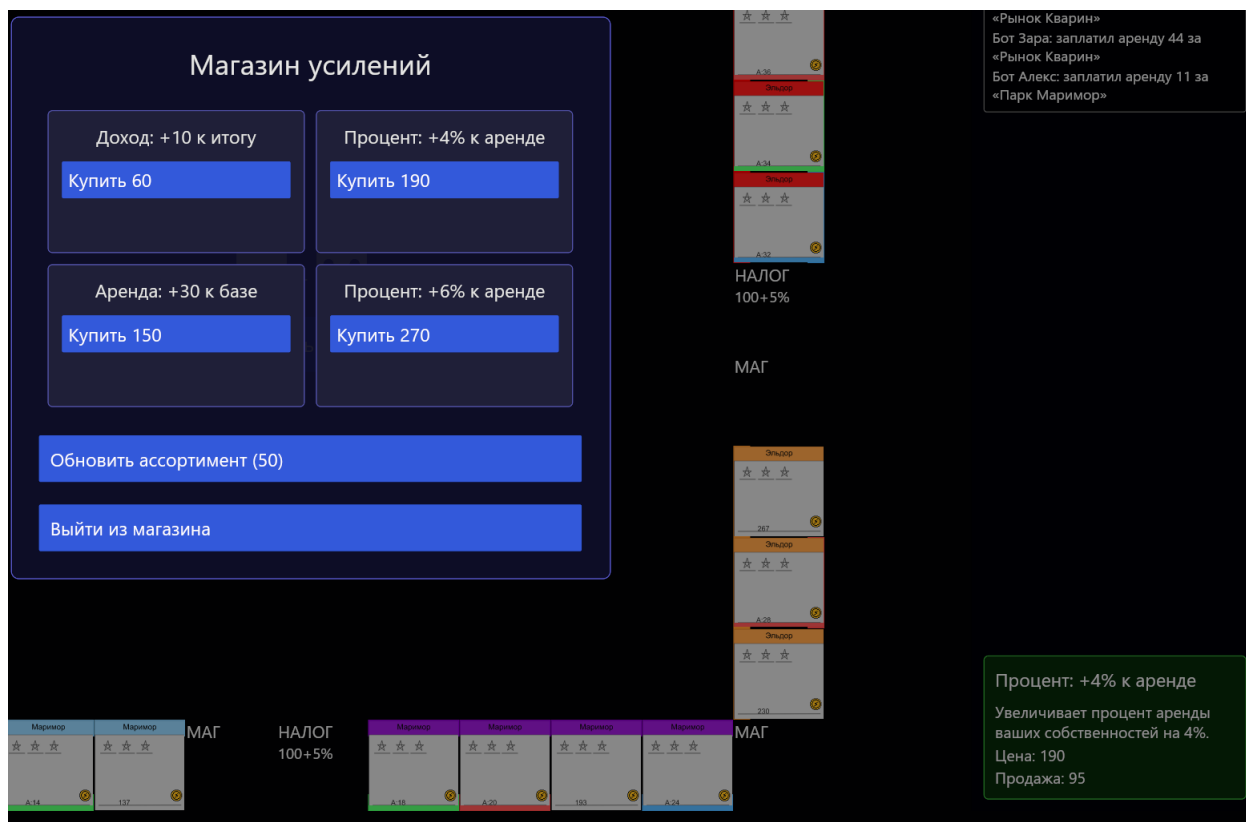


Рисунок 5.12 – Всплывающая подсказка усиления

При наведении на усиление в инвентаре или магазине появляется всплывающая подсказка с подробным описанием эффекта: тип (flat_base, percent_bonus или flat_final), числовое значение и стоимость.

На рисунке 5.13 представлена панель управления собственностью.

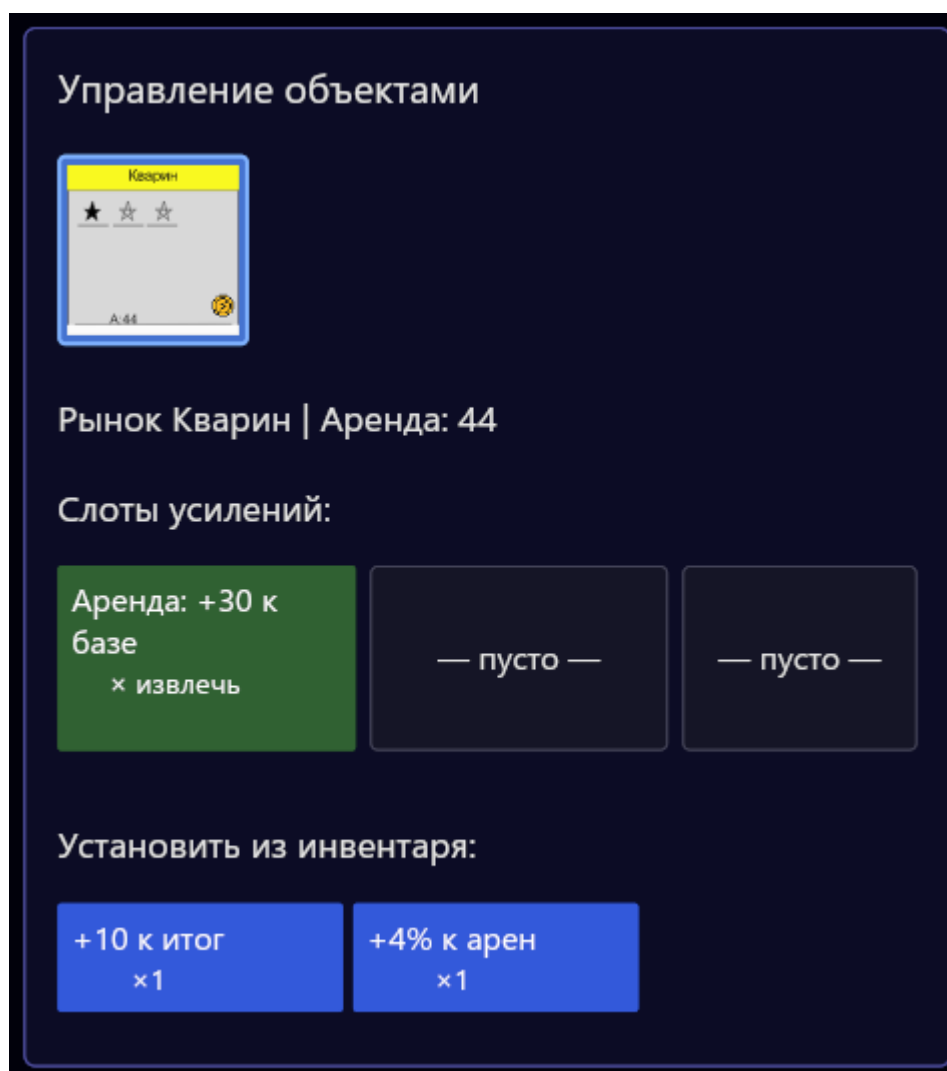


Рисунок 5.13 – Панель управления собственностью

Панель управления собственностью открывается по кнопке во время хода. В ней отображается список всех собственностей игрока с текущими слотами усиления. Игрок может установить усиление из инвентаря в свободный слот или извлечь его обратно. Каждая собственность поддерживает до трёх усилений одновременно.

На рисунке 5.14 представлена всплывающая подсказка клетки на игровом поле.

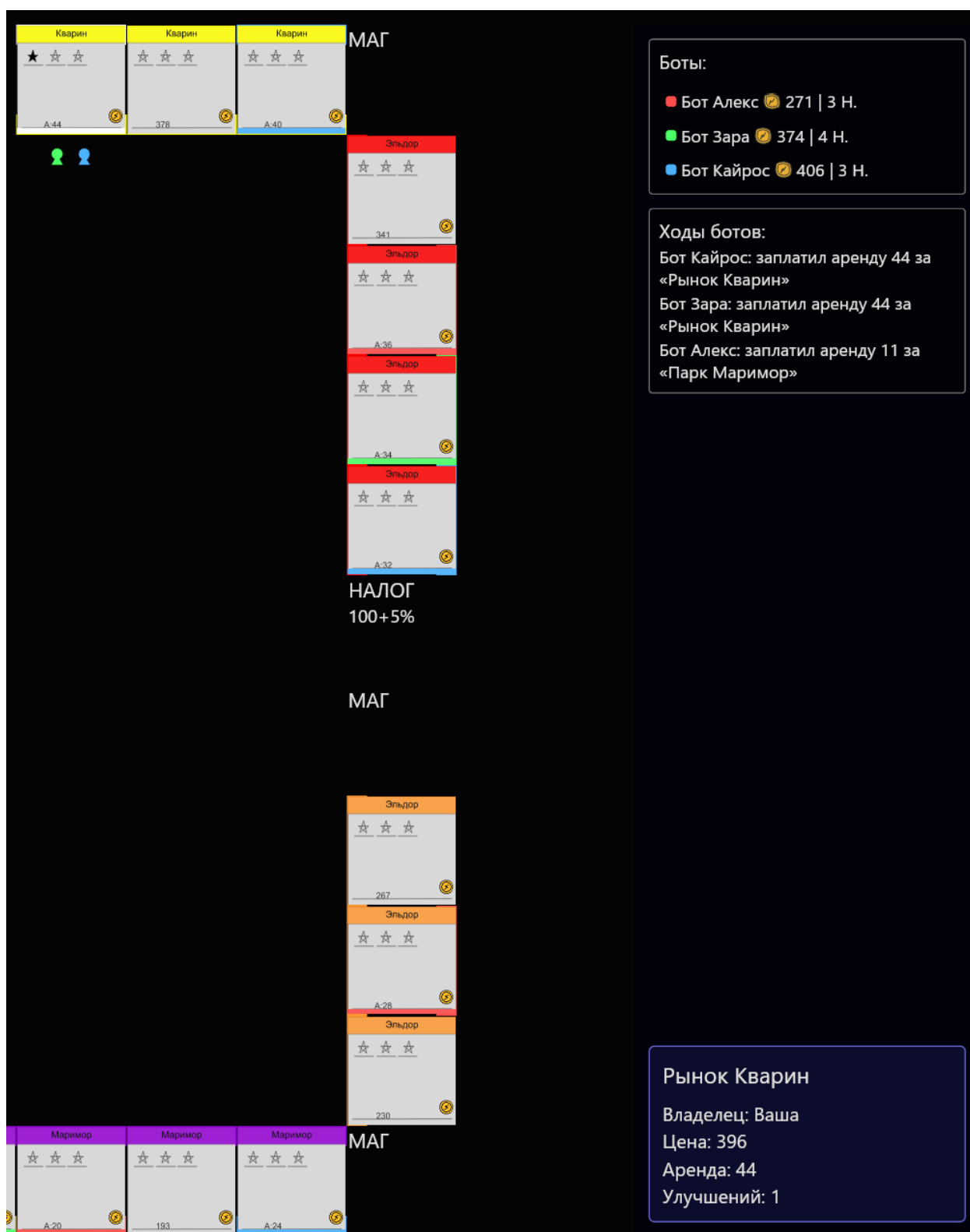


Рисунок 5.14 – Панель управления собственностью

При наведении курсора на клетку игрового поля появляется подсказка с информацией о клетке: для собственности отображается название, владелец, стоимость покупки, размер аренды и количество установленных усиления.

На рисунке 5.15 представлен экран статистики профиля.

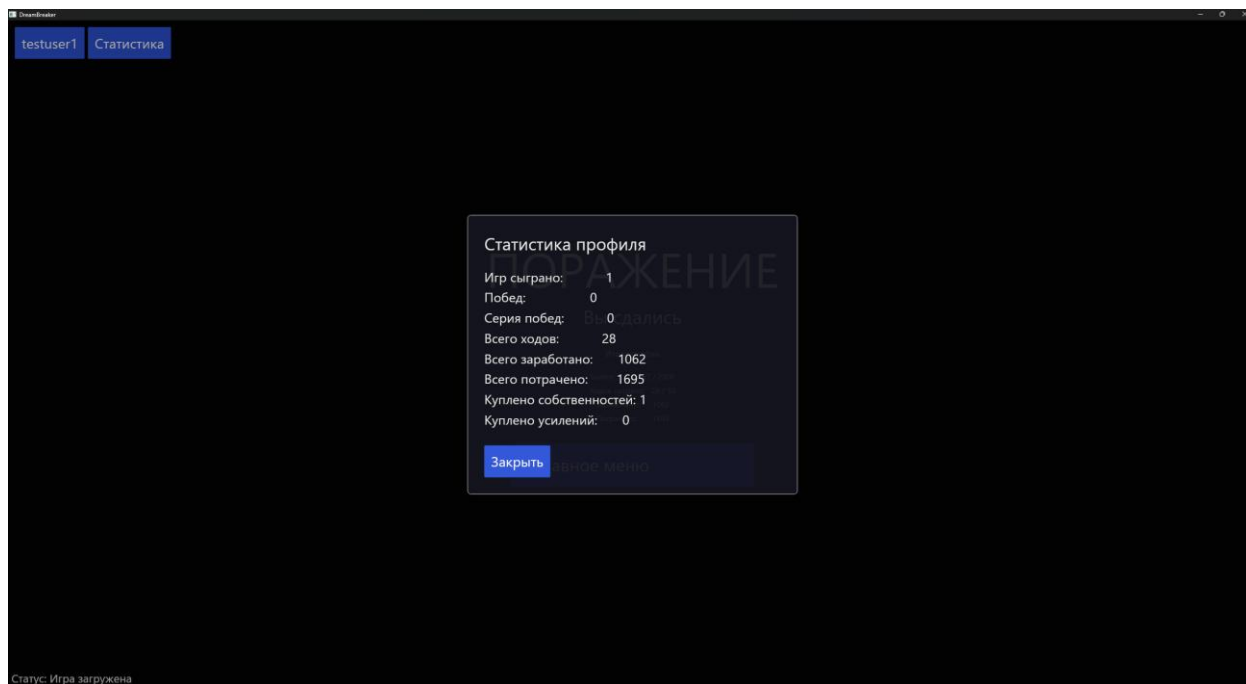


Рисунок 5.14 – Экран статистики профиля

Экран статистики открывается из главного меню. В нём отображаются накопленные показатели игрока по всем завершённым партиям: общее количество игр, количество побед, текущая серия побед, суммарно совершённые ходы, заработанные и потраченные средства, количество купленных собственности и усилений.

На рисунке 5.15 представлен экран завершения партии.

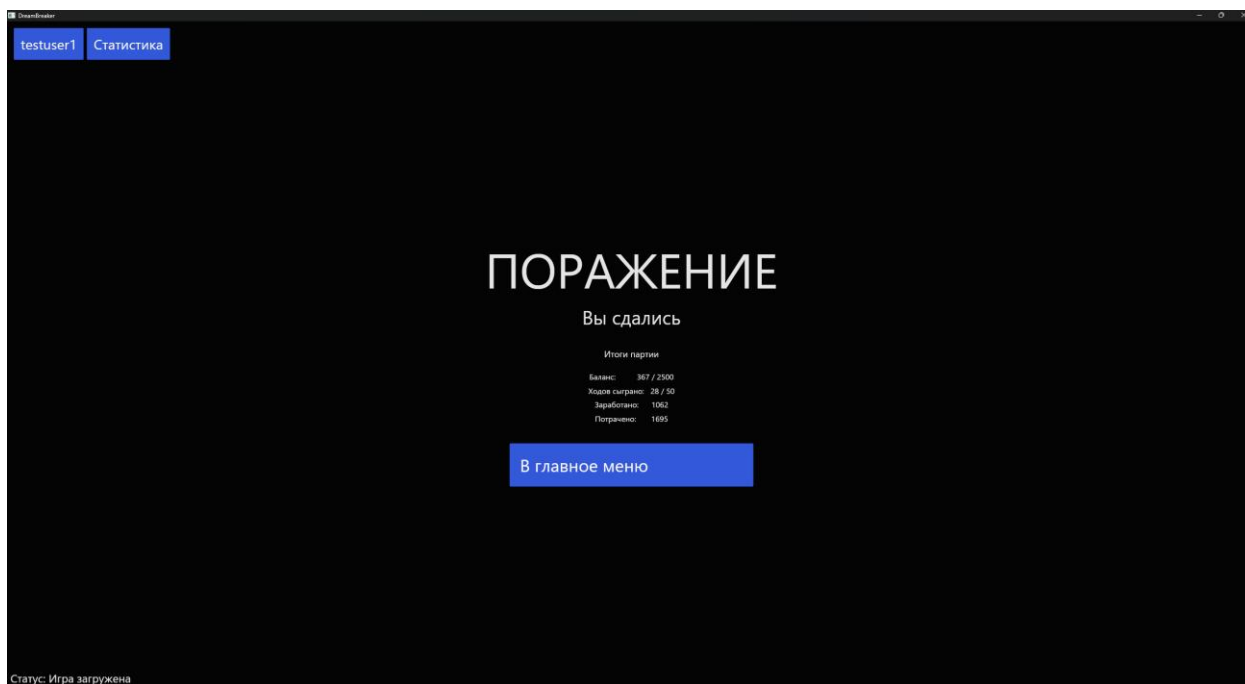


Рисунок 5.15 – Экран завершения партии

По завершении партии отображается экран с итогами: результат (победа или поражение), причина завершения, финальный баланс относительно целевого, количество сыгранных ходов, суммарно заработанные и потраченные средства.

На рисунке 5.16 представлен экран настроек.

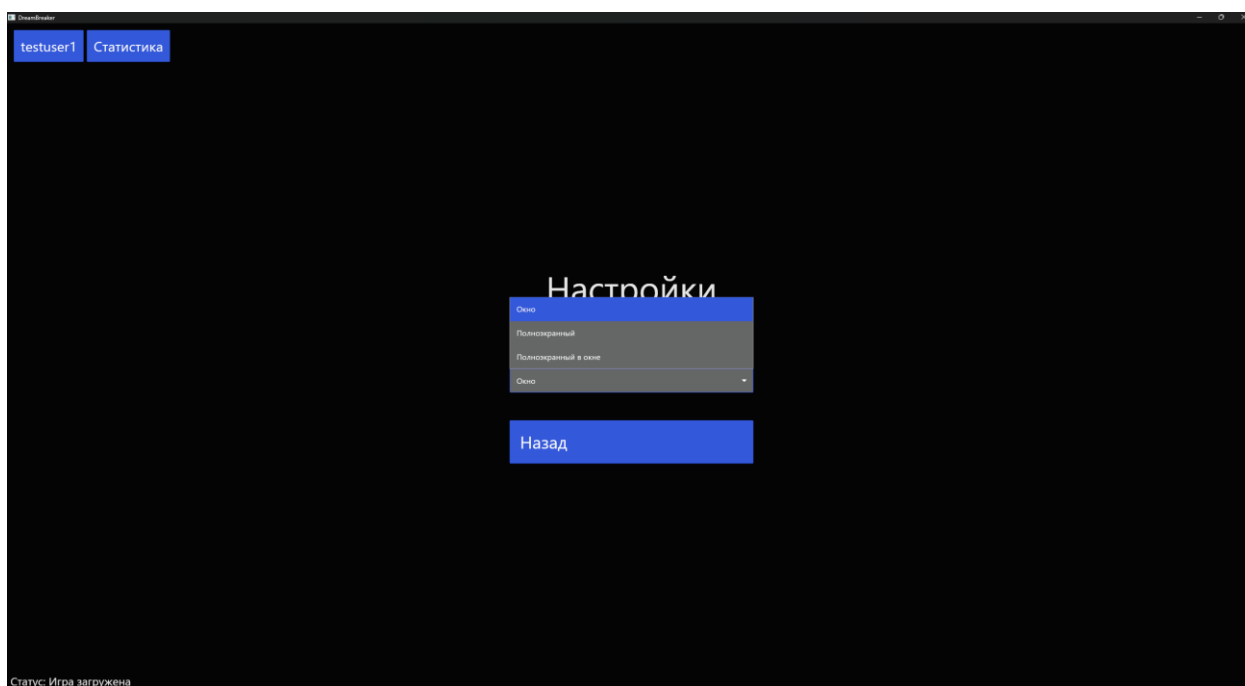


Рисунок 5.16 – Экран настроек

В разделе настроек пользователь выбирает режим отображения окна приложения из трёх доступных вариантов. Изменение применяется немедленно без перезапуска.

Таким образом, разработанный интерфейс обеспечивает выполнение всех игровых операций: регистрацию и вход, создание и загрузку партии, управление ходами, покупку и установку усилений, управление собственностью и просмотр статистики. Единый тёмный стиль оформления и автоматическое масштабирование интерфейса обеспечивают удобство работы при различных разрешениях экрана.

6 ТЕСТИРОВАНИЕ И ДОРАБОТКА СИСТЕМЫ

В процессе разработки автоматизированной системы «DreamBreaker» было проведено комплексное тестирование, направленное на проверку соответствия реализованного функционала требованиям технического задания. Основной акцент был сделан на ручном тестировании через графический интерфейс приложения.

6.1 Ручное тестирование

Для проведения тестирования были составлены тест-кейсы.

В таблице 6.1 представлен тест-кейс на попытку sql-инъекции в базу данных.

Таблица 6.1 – Тест-кейс на ввод sql-инъекции в базу данных

Параметр	Значение
ID	1
Название	SQL-Инъекция
Предусловие	1. Пользователь не авторизован 2. Пользователь не зарегистрирован
Входные параметры	Нет
Шаги	1. Открытие окна авторизации/регистрации 2. Ввод в поле логин некоторой SQL-Инъекции 3. Ввод в поле пароль любого пароля 4. Нажатие кнопки «Создать аккаунт»
Ожидаемый результат	1. Выполнение регистрации и вход в аккаунт 2. Отображение SQL-Инъекции как логина

В ходе тестирования тест-кейс SQL-Инъекций был проведен несколько раз. Ожидаемый результат был достигнут в каждом случае. Результат тестирования представлен в приложении Е.

В таблице 6.2 представлен тест-кейс на попытку авторизации с неверным паролем.

Таблица 6.2 – тест-кейс попытки авторизации при неверном пароле

Параметр	Значение
ID	2
Название	Авторизация с неверным паролем
Предусловие	1. Пользователь не авторизован 2. Пользователь зарегистрирован
Входные параметры	Логин существующего аккаунта
Шаги	1. Открытие окна авторизации/регистрации 2. Выбор аккаунта для входа 3. Ввод в поле пароль любого пароля, кроме верного 4. Нажатие кнопки «Войти»
Ожидаемый результат	1. Не успешная авторизация 2. Отображение ошибки

В ходе тестирования был достигнут ожидаемый результат. Результат тестирования представлен в приложении Е.

В таблице 6.3 представлен тест-кейс на попытку регистрации аккаунта с существующим логином.

Таблица 6.3 – Тест-кейс на попытку дублирования логина при авторизации

Параметр	Значение
ID	3
Название	Дубликат логина
Предусловие	1. Пользователь не авторизован 2. Пользователь не зарегистрирован
Входные параметры	1. Существующий аккаунт с логином и паролем 2. Вводимое значение логина аналогично существующему
Шаги	1. Открытие окна авторизации/регистрации 2. Ввод в поле логин существующего логина 3. Ввод в поле пароль любого пароля 4. Нажатие кнопки «Создать аккаунт»
Ожидаемый результат	1. Не успешная попытка регистрации нового аккаунта 2. Отображение ошибки

В ходе тестирования был достигнут ожидаемый результат. Результат тестирования представлен в приложении Е.

В таблице 6.4 представлен тест-кейс на попытку успешной авторизации пользователя.

Таблица 6.4 – Тест-кейс на попытку успешной авторизации

Параметр	Значение
ID	4
Название	Успешная авторизация
Предусловие	1. Пользователь не авторизован 2. Пользователь зарегистрирован
Входные параметры	1. Логин существующего аккаунта 2. Пароль существующего аккаунта
Шаги	1. Открытие окна авторизации/регистрации 2. Выбор аккаунта для входа 3. Ввод в поле «Пароль» правильного пароля 4. Нажатие кнопки «Войти»
Ожидаемый результат	1. Успешная авторизация 2. Отображение статуса успешной авторизации

В ходе тестирования был достигнут ожидаемый результат. Результат тестирования представлен в приложении Е.

В таблице 6.5 представлен тест-кейс успешного создания новой игры.

Таблица 6.5 – Тест-кейс успешного создания новой игры

Параметр	Значение
ID	5
Название	Успешное создание игры
Предусловие	1. Пользователь авторизован 2. Пользователь зарегистрирован 3. Пользователь выбрал любые параметры новой игры
Входные параметры	Входные параметры новой игры
Шаги	1. Нажатие на кнопку «Создать новую игру» 2. Выбор любых параметров новой игры. 3. Нажатие на кнопку «Создать игру»
Параметр	Значение
Ожидаемый результат	1. Успешное создание новой партии 2. Отображение статуса успешного создания новой игры. 3. Отображение игрового поля

В ходе тестирования был достигнут ожидаемый результат. Результат тестирования представлен в приложении Е.

В таблице 6.6 представлен тест-кейс успешной загрузки сохранённой не завершённой игры.

Таблица 6.6 – Тест-кейс успешной загрузки сохранённой не завершённой игры

Параметр	Значение
ID	6
Название	Успешная загрузка игры
Предусловие	1. Пользователь авторизован 2. Пользователь зарегистрирован 3. Пользователь обладает играми со статусом «Приостановлена» или «Активна»
Входные параметры	Нет
Шаги	1. Нажатие на кнопку «Загрузить игру» 2. Выбор игры со статусом «Приостановлена» или «Активна» 3. Нажатие на кнопку «Загрузить»
Ожидаемый результат	1. Успешная загрузка партии 2. Отображение статуса успешной загрузки игры. 3. Отображение игрового поля

В ходе тестирования был достигнут ожидаемый результат. Результат тестирования представлен в приложении Е.

В ходе проведения ручного тестирования не было выявлено ни одной ошибки. Функции игровой логики были проверены при функциональном тестировании.

6.2 Функциональное тестирование

Функциональное тестирование было проведено для всех функций игрового процесса приложения «DreamBreaker». Тестирование не требует запуска графического интерфейса и выполняется путём прямого вызова хранимых функций PostgreSQL через отдельное тестовое подключение.

Для функционального тестирования были выбраны следующие хранимые функции:

1. register_new_user – регистрация нового пользователя;
2. get_user_credentials – получение хеша пароля и статуса по имени пользователя;
3. get_user_by_id – получение данных пользователя по UUID;
4. list_users – список всех активных human-пользователей;
5. create_game_full – создание игры с полем, правилами, участниками и ботами за одну транзакцию;
6. pause_game – сохранение и выход из игры (статус paused);
7. surrender_game – сдача игры (статус surrender);
8. set_game_status – принудительная смена статуса игры;
9. get_user_games – список всех игр пользователя;
10. get_latest_user_game – последняя активная или приостановленная игра пользователя;
11. get_active_game – UUID текущей активной игры пользователя;
12. get_active_game_for_user – алиас для get_active_game;
13. user_has_active_game – проверка наличия активной игры у пользователя;
14. list_games – список всех игр в системе;
15. get_game_rules – правила конкретной партии;
16. list_game_participants – список всех участников игры;
17. reset_stale_active_games – перевод всех лишних активных игр пользователя в paused, кроме последней;

18. `commit_player_move` – фиксация хода игрока: обновление позиции, начисление бонуса за старт;
19. `get_board_cells` – все 40 клеток игрового поля с данными собственностей и магазинов;
20. `get_participant_state` – текущее состояние участника: позиция, баланс, ходы, статистика;
21. `calc_rent` – расчёт аренды собственности с учётом усилений владельца;
22. `pay_tax` – списание налога ($100 + 5\%$ от баланса)
23. `pay_rent` – оплата аренды: списание у арендатора, зачисление владельцу;
24. `buy_property` – покупка собственности игроком;
25. `get_shop_slots` – слоты магазина с данными усилений;
26. `buy_shop_slot` – покупка усиления из магазина;
27. `reroll_shop` – обновление ассортимента магазина;
28. `list_power_ups` – справочник всех усилений;
29. `add_to_inventory` – добавление усиления в инвентарь игрока;
30. `get_player_inventory` – инвентарь игрока в конкретной партии;
31. `sell_power_up` – продажа усиления из инвентаря за половину стоимости;
32. `install_upgrade` – установка усиления на собственность;
33. `uninstall_upgrade` – снятие усиления с собственности в инвентарь;
34. `get_player_properties` – список собственностей игрока с данными об усилениях;
35. `get_all_balances` – балансы всех участников игры;
36. `get_user_stats` – статистика профиля по завершённым играм;
37. `get_game_result` – итоги завершённой партии: баланс, ходы, победа/поражение;
38. `get_bot_participants` – список активных ботов в игре;

39. `do_bot_turn` – выполнение хода бота: движение, покупка, аренда, налог, банкротство.

Для функций, допускающих граничные и ошибочные входные данные, тестирование проводилось в двух сценариях: с корректными входными данными и с данными, которые должны вызвать отказ или вернуть пустой результат.

Результаты функционального тестирования представлены на рисунке 6.1. В ходе проведения тестирования были выявлены некоторые ошибки в работе программы, которые были исправлены позднее.

```
Администратор: Windows Po X + v
Applied 20/migrate stats and game actions (653.4µs)
PS C:\Dreambreaker\dreambreaker_tests> cargo test -- --test-threads=1
warning: unused manifest key: package.doctest
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.23s
Running unittests src\lib.rs (target\debug\deps\dreambreaker_tests-2e652a62ca45f331.exe)

running 58 tests
test tests_auth::test_get_credentials_existing ... ok
test tests_auth::test_get_credentials_nonexistent ... ok
test tests_auth::test_register_duplicate_user ... ok
test tests_auth::test_register_new_user ... ok
test tests_board::test_calc_rent_no_owner ... ok
test tests_board::test_calc_rent_with_owner ... ok
test tests_board::test_get_board_cells ... ok
test tests_board::test_get_participant_state ... ok
test tests_board::test_pay_tax ... ok
test tests_bots::test_do_bot_turn_updates_position ... ok
test tests_bots::test_get_bot_participants ... ok
test tests_bots::test_get_player_properties ... ok
test tests_gameplay::test_buy_property_decreases_balance ... ok
test tests_gameplay::test_commit_player_move_updates_position ... ok
test tests_gameplay::test_pay_rent_transfers_balance ... ok
test tests_gameplay::test_start_bonus_on_pass ... ok
test tests_games::test_board_has_40_cells ... ok
test tests_games::test_create_game_custom_params ... ok
test tests_games::test_create_game_default ... ok
test tests_games::test_load_paused_game ... ok
test tests_games_extended::test_get_active_game ... ok
test tests_games_extended::test_get_game_rules ... ok
test tests_games_extended::test_get_latest_user_game ... ok
test tests_games_extended::test_get_user_games ... ok
test tests_games_extended::test_list_game_participants ... ok
test tests_games_extended::test_list_games ... ok
test tests_games_extended::test_set_game_status ... ok
test tests_games_extended::test_user_has_active_game_false ... ok
test tests_games_extended::test_user_has_active_game_true ... ok
test tests_misc::test_get_active_game_for_user ... ok
test tests_misc::test_update_game_timestamp_trigger ... ok
test tests_power_ups_extended::test_add_to_inventory ... ok
test tests_power_ups_extended::test_add_to_inventory_accumulates ... ok
test tests_power_ups_extended::test_get_player_inventory ... ok
test tests_power_ups_extended::test_list_power_ups ... ok
test tests_power_ups_extended::test_sell_power_up ... ok
test tests_power_ups_extended::test_sell_power_up_not_in_inventory ... ok
test tests_reset_stale::test_reset_stale_active_games ... ok
test tests_shop::test_buy_power_up_adds_to_inventory ... ok
test tests_shop::test_buy_power_up_decreases_balance ... ok
test tests_shop::test_buy_power_up_insufficient_balance ... ok
test tests_shop::test_shop_has_slots ... ok
test tests_shop_extended::test_get_shop_slots ... ok
test tests_shop_extended::test_reroll_shop ... ok
test tests_stats::test_get_all_balances ... ok
test tests_stats::test_get_game_result_surrender ... ok
test tests_stats::test_get_game_result_victory ... ok
test tests_stats::test_get_user_stats_after_surrender ... ok
test tests_stats::test_get_user_stats_empty ... ok
test tests_stats::test_surrender_game ... ok
test tests_stats::test_surrender_game_not_participant ... ok
test tests_upgrades::test_install_upgrade ... ok
test tests_upgrades::test_install_upgrade_removes_from_inventory ... ok
test tests_upgrades::test_install_upgrade_slot_limit ... ok
test tests_upgrades::test_uninstall_upgrade_returns_to_inventory ... ok
test tests_users_extended::test_get_user_by_id ... ok
test tests_users_extended::test_get_user_by_id_nonexistent ... ok
test tests_users_extended::test_list_users_returns_humans_only ... ok

test result: ok. 58 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 6.85s

Doc-tests dreambreaker_tests
```

Рисунок 6.1 – Результаты функционального тестирования

7 ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Руководство пользователя содержит описание интерфейса, игрового процесса, системных требований и возможных технических проблем с путями их решения. Полный текст руководства расположен в директории «техническая документация» в корне проекта. Ссылки на репозитории приведены в приложении Д.

Заключение

В ходе выполнения данной курсовой работы была разработана автоматизированная система «DreamBreaker», представляющая собой десктопное приложение стратегической и экономической игры в жанре настольной игры с элементами монополии.

В ходе работы были выполнены следующие задачи:

1. Составлено технического задания;
2. Проведена оценка рисков;
3. Разработана структурная, функциональная и информационная модели системы;
4. Спроектирована база данных;
5. Спроектирована серверная часть приложения;
6. Спроектирована клиентская часть приложения;
7. Разработана и интегрирована серверной и клиентской частей;
8. Проведено тестирование и доработка системы;
9. Написано руководство пользователя и техническая документация;
10. Подготовлена пояснительная записка и отчет о проделанной работе.

Результаты тестирования показали, что система корректно выполняет заявленные функции. Все 58 автоматизированных тестов пройдены успешно: функции регистрации и авторизации, создания и управления игровыми сессиями, выполнения ходов, расчёта аренды и налогов, работы магазина усиления и подсчёта статистики работают в соответствии с требованиями. Ручное тестирование пользовательского интерфейса подтвердило корректность отображения игрового поля, работу диалогов покупки собственности и усиления, а также корректное завершение игровых сессий.

Таким образом, разработанная автоматизированная система «DreamBreaker» соответствует требованиям технического задания, обеспечивает реализацию полного игрового цикла и может быть использована

для организации досуга пользователя в формате стратегической экономической игры. Ссылки на репозитории проекта приведены в приложении Д.

Список использованных источников

1. ОС ТУСУР 01-2021 [Электронный ресурс]: сайт tusur.ru. URL: <https://regulations.tusur.ru/documents/70> (дата обращения: 11.06.2025).
2. Петрова И.Р., Фахртдинов Р.Х., Сулейманова А.А. Методология функционального моделирования IDEF0 [Электронный ресурс]: учебно-методическое пособие / Казанский федеральный университет. URL: https://kpfu.ru/staff_files/F1506701798/Metodichka_IDEF.pdf (дата обращения: 09.04.2026).
3. Сущности и связи: как и для чего системные аналитики создают ER-диаграммы [Электронный ресурс]: сайт practicum.yandex.ru. URL: <https://practicum.yandex.ru/blog/chto-takoe-er-diagramma/> (дата обращения: 09.04.2026).
4. Class Diagram Tutorial [Электронный ресурс]: сайт Visual Paradigm. URL: <https://online.visual-paradigm.com/diagrams/tutorials/class-diagram-tutorial/> (дата обращения: 09.04.2026).
5. Диаграмма потоков данных (DFD) для чайников: что это такое, как сделать и какой бывает [Электронный ресурс]: сайт habr.com. URL: <https://habr.com/ru/companies/kaiten/articles/925044/> (дата обращения: 11.06.2026).
6. PostgreSQL Documentation [Электронный ресурс]: сайт postgresql.org. URL: <https://www.postgresql.org/docs/> (дата обращения: 01.04.2026).
7. Руководство по языку программирования Rust [Электронный ресурс]: сайт metanit.com. URL: <https://metanit.com/rust/tutorial/> (дата обращения: 07.04.2026).
8. iced – Rust [Электронный ресурс]: сайт docs.iced.rs. URL: <https://docs.iced.rs/iced/> (дата обращения: 07.04.2026).

Приложение А

(Обязательное)

Оценка рисков Репьюк Н. С.

Название — описание — условие возникновения — последствия — связанные риски — вероятность риска(числ.[1-5]) — критичность последствий(числ.[1-5])

№	Название	Описание	Условие возникновения	Последствия	Связанные риски	Вероятность риска	Критичность последствий
1	Ошибки в работе продукта, не полная/не качественная реализация функционала, указанного в тз	Некорректная работа продукта вследствие отсутствия приоритета качества	Смещение приоритета с качества продукта на иные его аспекты	Накопление технического долга в угоду срокам	3, 5, 12	4	2
2	Утрата результатов деятельности	Потеря кода, документации или схем базы данных вследствие сбоя оборудования или человеческой ошибки	Работа ведется на личных ресурсах, отсутствие автоматического бекапа, случайное удаление файлов.	Потери данных разработки	-	1	6
3	Неверное математическое обоснование	Некорректная работа экономических расчетов, нарушение целостности данных при сохранении/загрузке, некорректные связи между таблицами	Недостаточное тестирование математических моделей	Ошибки функционирования архитектуры БД и игровой логики	1, 11	5	4
4	Компроме	Различны	Недостато	Проблемы	7,8		

Рисунок А.1 – Оценка рисков Репьюк Н.С.

	тация учетных данных пользователей, несоответствие формальным требованиям ТЗ	е нарушения в безопасности данных, в т.ч. нарушения пунктов 4.3, 4.4, 4.6 тз	чное внимание к вопросам безопасности при разработке	с безопасностью данных		2	7
5	Нестабильность и низкая производительность приложения	Приложение "вылетает", зависает или сильно грузит систему при выполнении игровых действий	Недостаточная оптимизация действий, создание лишней сущностей, использования большого количества временных решений	Негативный пользовательский опыт	1,9,12	5	3
6	Использование программного продукта не по прямому назначению	Отсутствие или недостаточность документации	Недооценка важности этапа документирования	Снижение оценки, не допуск к защите проекта	-	2	1
7	Компрометация базы данных	SQL-инъекции в подсистеме аутентификации	Отсутствие параметризованных запросов или экранирования пользовательского ввода при взаимодействии с PostgreSQL.	Получение несанкционированного доступа к учётным записям, кража или модификация данных пользователей, удаление таблиц,	4, 8	4	7

Рисунок А.2 – Оценка рисков Репьюк Н.С.

				компрометация всей системы.			
8	НСД к системе	Подбор учётных данных (логинов и паролей) для несанкционированного входа в систему.	Отсутствие механизмов защиты: ограничена численность попыток входа, временных блокировок, журналирования неудачных попыток.	Взлом учётных записей пользователей, кража игровых данных, возможность совершения действий от имени другого пользователя.	4, 7	2	6
9	Недоступность системы для легитимных пользователей	Генерация большого количества запросов к системе с целью исчерпания ресурсов сервера.	Отсутствия ограничений на частоту операций со стороны клиента, недостаточная защита от автоматизации.	Отказ в обслуживании (DoS) через исчерпание ресурсов	5, 8	5	5
10	Утрата игрового прогресса, подмена данных, неработоспособность приложения.	Пользователь получает прямой доступ к файлам БД, минуя приложение, и изменяет или удаляет их.	Хранение данных в незашифрованном виде, отсутствие проверки целостности, запуск приложения на заражённой машине	Компрометация локальной базы данных через файловую систему	2, 10	2	6
11	Нарушение	Пользователь	Отсутствия	Нечестная	3		

Рисунок А.3 – Оценка рисков Репьюк Н.С.

	е игрового баланса, несоответствие ожидаемому поведению системы, сложность отладки.	ель намеренно изменяет файлы сохранения и/или напрямую БД, чтобы получить неограниченные ресурсы, обойти игровые условия и т.п.	е проверки целостности игровых данных, подписи сохранения и/или шифрования.	игра через редактирование сохранения		6	2
12	Некорректная обработка специальных символов	Ввод пользователем специальных символов в поля, которые неправильно обрабатываются при формировании запросов или отображении, что может вызвать сбой или повреждение данных.	Отсутствие экранирования ввода, использование конкатенации строк в SQL или при работе с файлами.	Ошибки выполнения SQL, аварийное завершение приложения, порча данных.	3, 8	6	3

ФИО

специалиста Репьюк Наталья Сергеевна

Образование высшее

Организация ТУСУР, КИБЭВС

Стаж работы 5

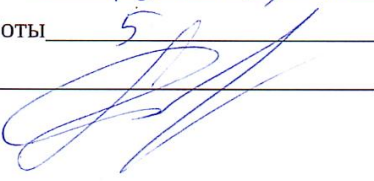
Подпись 

Рисунок А.4 – Оценка рисков Репьюк Н.С.

Приложение Б

(Обязательное)

Оценка рисков Мацкевич Е. В.

Название — описание — условие возникновения — последствия — связанные риски — вероятность риска(числ.[1-5]) — критичность последствий(числ.[1-5])

№	Название	Описание	Условие возникновения	Последствия	Связанные риски	Вероятность риска	Критичность последствий
1	Ошибки в работе продукта, не полная/не качественная реализация функционала, указанного в тз	Некорректная работа продукта вследствие отсутствия приоритета качества	Смещение приоритета с качества продукта на иные его аспекты	Накопление технического долга в угоду срокам	3, 5, 12	2	7
2	Утрата результата в деятельности	Потеря кода, документации или схем базы данных вследствие сбоя оборудования или человеческой ошибки	Работа ведется на личных ресурсах, отсутствие автоматического бекапа, случайное удаление файлов.	Потери данных разработки	-	2	8
3	Неверное математическое обоснование	Некорректная работа экономических расчетов, нарушение целостности данных при сохранении/загрузке, некорректные связи между таблицами	Недостаточное тестирование математических моделей	Ошибки функционирования архитектуры БД и игровой логики	1, 11	5	9
4	Компроме	Различны	Недостато	Проблемы	7,8		

Рисунок Б.1 – Оценка рисков Мацкевич Е. В.

	тация учетных данных пользоват елей, несоответ ствие формальн ым требовани ям ТЗ	е нарушени я в безопасно сти данных, в т.ч. нарушени е пунктов 4.3, 4.4, 4.6 тз	чное внимание к вопросам безопасно сти при разработк е	с безопасно стью данных		1	5
5	Нестабиль ность и низкая производи тельность приложен ия	Приложен ие "вылетает ", зависает или сильно грузит систему при выполнен ии игровых действий	Недостато чная оптимизац ия действий, создание лишних сущности й, использов ания большого количеств а временны х решений	Негативн ый пользоват ельский опыт	1,9 ,12	5	3
6	Использов ание программ ного продукта не по прямому назначени ю	Отсутстви е или недостато чность документа ции	Недооцен ка важности этапа документи рования	Снижение оценки, не допуск к защите проекта	-	2	3
7	Компроме тация базы данных	SQL- инъекции в подсистем е аутентифи кации	Отсутстви е параметри зованных запросов или экраниров ания пользоват ельского ввода при взаимодей ствии с PostgreSQL.	Получени е несанкцио нированно го доступа к учётным записям, кража или модифика ция данных пользоват елей, удаление таблиц,	4, 8	2	10

Рисунок Б.2 – Оценка рисков Мацкевич Е. В.

				компрометация всей системы.			
8	НСД к системе	Подбор учётных данных (логинов и паролей) для несанкционированного входа в систему.	Отсутствие механизмов защиты: ограничения числа попыток входа, временных блокировок, журналирования неудачных попыток.	Взлом учётных записей пользователей, кража игровых данных, возможность совершения действий от имени другого пользователя.	4, 7	2	7
9	Недоступность системы для легитимных пользователей	Генерация большого количества запросов к системе с целью исчерпания ресурсов сервера.	Отсутствие ограничений на частоту операций со стороны клиента, недостаточная защита от автоматизации.	Отказ в обслуживании (DoS) через исчерпание ресурсов	5, 8	2	6
10	Утрата игрового прогресса, подмена данных, неработоспособность приложения.	Пользователь получает прямой доступ к файлам БД, минуя приложение, и изменяет или удаляет их.	Хранение данных в незашифрованном виде, отсутствие проверки целостности, запуск приложения на заражённой машине	Компрометация локальной базы данных через файловую систему	2, 10	7	2
11	Нарушения	Пользователь	Отсутствия	Нечестная	3		

Рисунок Б.3 – Оценка рисков Мацкевич Е. В.

	е игрового баланса, несоответствие ожидаемому поведению системы, сложность отладки.	ель намеренно изменяет файлы сохранения и/или напрямую БД, чтобы получить неограниченные ресурсы, обойти игровые условия и т.п.	е проверки целостности игровых данных, подписи сохранения и/или шифрования.	игра через редактирование сохранений		6	4
12	Некорректная обработка специальных символов	Ввод пользователем специальных символов в поля, которые неправильно обрабатываются при формировании запросов или отображении, что может вызвать сбой или повреждение данных.	Отсутствие экранирования ввода, использование конкатенации строк в SQL или при работе с файлами.	Ошибки выполнения SQL, аварийное завершение приложения, порча данных.	3, 8	8	5

ФИО

специалиста Мацкевич Елена Викторовна

Образование высшее

Организация ТУСУР КИБЭВС

Стаж работы 12 лет

Подпись ЕВМ

Рисунок Б.4 – Оценка рисков Мацкевич Е. В.

Приложение В

(Обязательное)

Оценка рисков Мальчукова А. Н.

Название — описание — условие возникновения — последствия — связанные риски — вероятность риска(числ.[1-5]) — критичность последствий(числ.[1-5])

№	Название	Описание	Условие возникновения	Последствия	Связанные риски	Вероятность риска	Критичность последствий
1	Ошибки в работе продукта, не полная/не качественная реализация функционала, указанного в тз	Не корректная работа продукта вследствие отсутствия приоритета качества	Смещение приоритета с качества продукта на иные его аспекты	Накопление технического долга в угоду срокам	3, 5, 12	3	4
2	Утрата результатов деятельности	Потеря кода, документации или схем базы данных вследствие сбоя оборудования или человеческой ошибки	Работа ведется на личных ресурсах, отсутствие автоматического бекапа, случайное удаление файлов.	Потери данных разработки	-	2	3
3	Неверное математическое обоснование	Некорректная работа экономических расчетов, нарушение целостности данных при сохранении/загрузке, некорректные связи между таблицами	Недостаточное тестирование математических моделей	Ошибки функционирования архитектуры БД и игровой логики	1, 11	2	3
4	Компроме	Различны	Недостато	Проблемы	7,8		

Рисунок В.1 – Оценка рисков Мальчукова А. Н.

	тация учетных данных пользоват елей, несоответ ствие формальн ым требовани ям ТЗ	е нарушени я в безопасно сти данных, в т.ч. нарушени е пунктов 4.3, 4.4, 4.6 тз	чное внимание к вопросам безопасно сти при разработк е	с безопасно стью данных		2	2
5	Нестабиль ность и низкая производи тельность приложен ия	Приложен ие "вылетает ", зависит или сильно грузит систему при выполнен ии игровых действий	Недостато чная оптимизац ия действий, создание лишних сущности й, использов ания большого количеств а временны х решений	Негативн ый пользоват ельский опыт	1,9 ,12	2	1
6	Использов ание программ ного продукта не по прямому назначени ю	Отсутстви е или недостато чность документа ции	Недооцен ка важности этапа документи рования	Снижение оценки, не допуск к защите проекта	-	1	1
7	Компроме тация базы данных	SQL- инъекции в подсистем е аутентифи кации	Отсутстви е параметри зованных запросов или экраниров ания пользоват ельского ввода при взаимодей ствии с PostgreSQL.	Получени е несанкцио нированно го доступа к учётным записям, кража или модифика ция данных пользоват елей, удаление таблиц,	4, 8	2	3

Рисунок В.2 – Оценка рисков Мальчукова А. Н.

				компрометация всей системы.			
8	НСД к системе	Подбор учётных данных (логинов и паролей) для несанкционированного входа в систему.	Отсутствие механизмов защиты: ограничения числа попыток входа, временных блокировок, журналирования неудачных попыток.	Взлом учётных записей пользователей, кража игровых данных, возможность совершения действий от имени другого пользователя.	4, 7	2	2
9	Недоступность системы для легитимных пользователей	Генерация большого количества запросов к системе с целью исчерпания ресурсов сервера.	Отсутствие ограничений на частоту операций со стороны клиента, недостаточная защита от автоматизации.	Отказ в обслуживании (DoS) через исчерпание ресурсов	5, 8	1	1
10	Утрата игрового прогресса, подмена данных, неработоспособность приложения.	Пользователь получает прямой доступ к файлам БД, минуя приложение, и изменяет или удаляет их.	Хранение данных в незашифрованном виде, отсутствие проверки целостности, запуск приложения на заражённой машине	Компрометация локальной базы данных через файловую систему	2, 10	2	2
11	Нарушение	Пользователь	Отсутствие	Нечестная	3		

Рисунок В.3 – Оценка рисков Мальчукова А. Н.

	е игрового баланса, несоответствие ожидаемому поведению системы, сложность отладки.	ель намеренно изменяет файлы сохранения и/или напрямую БД, чтобы получить неограниченные ресурсы, обойти игровые условия и т.п.	е проверки целостности игровых данных, подписи сохранения и/или шифрования.	игра через редактирование сохранения		1	1
12	Некорректная обработка специальных символов	Ввод пользователем специальных символов в поля, которые неправильно обрабатываются при формировании запросов или отображении, что может вызвать сбой или повреждение данных.	Отсутствие экранирования ввода, использование конкатенации строк в SQL или при работе с файлами.	Ошибки выполнения SQL, аварийное завершение приложения, порча данных.	3, 8	1	2

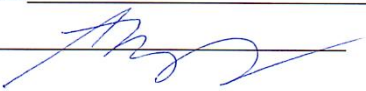
ФИО Мальчуков Андрей Николаевич
 специалиста _____
 Образование магистр по направлению «Информатика и физ. техника»
 Организация ТУСУР
 Стаж работы 20
 Подпись 

Рисунок В.4 – Оценка рисков Мальчукова А. Н.

Приложение Г (Обязательное)

Диаграммы и декомпозиции диаграмм

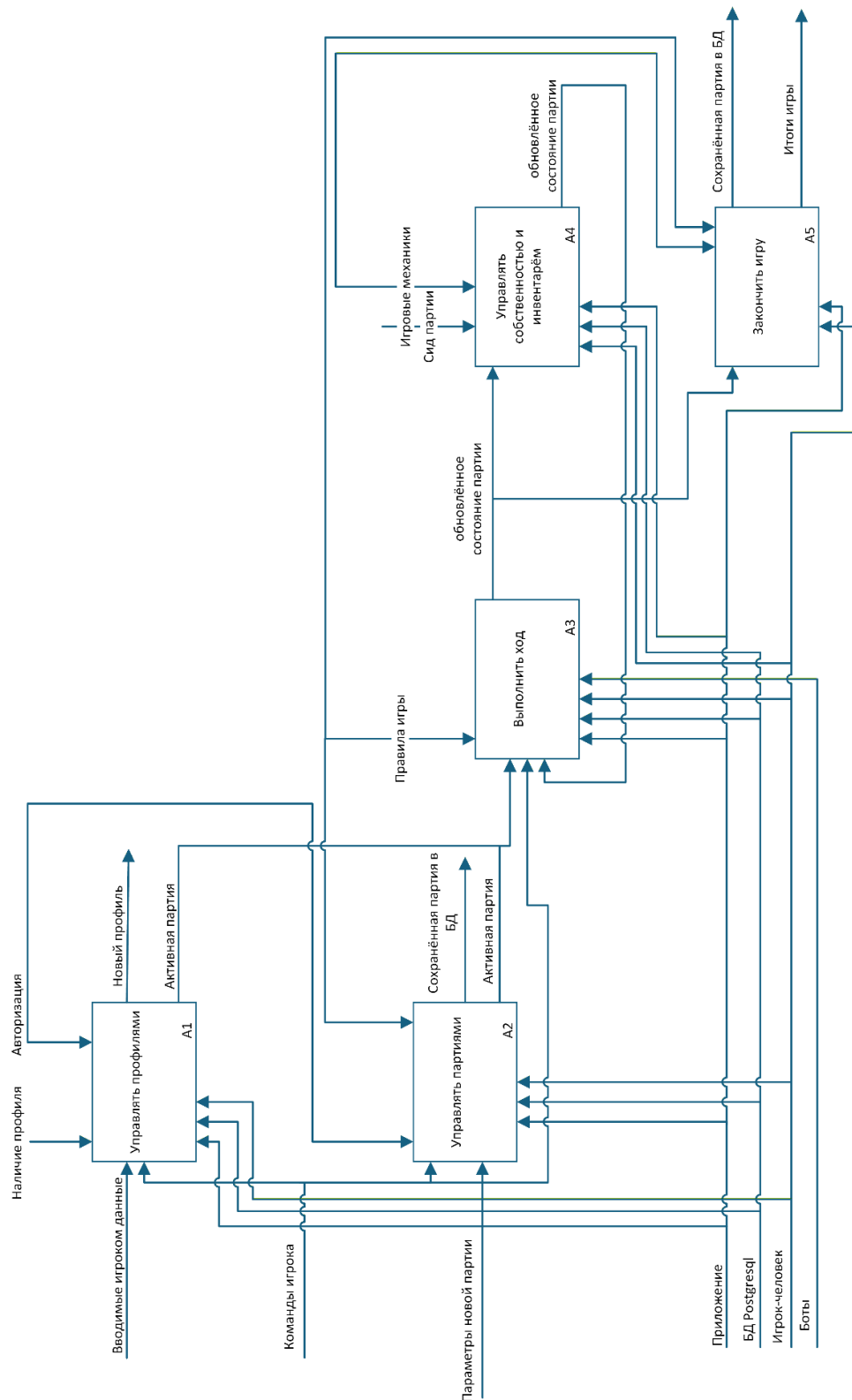


Рисунок Г.1 – Декомпозиция чёрного ящика

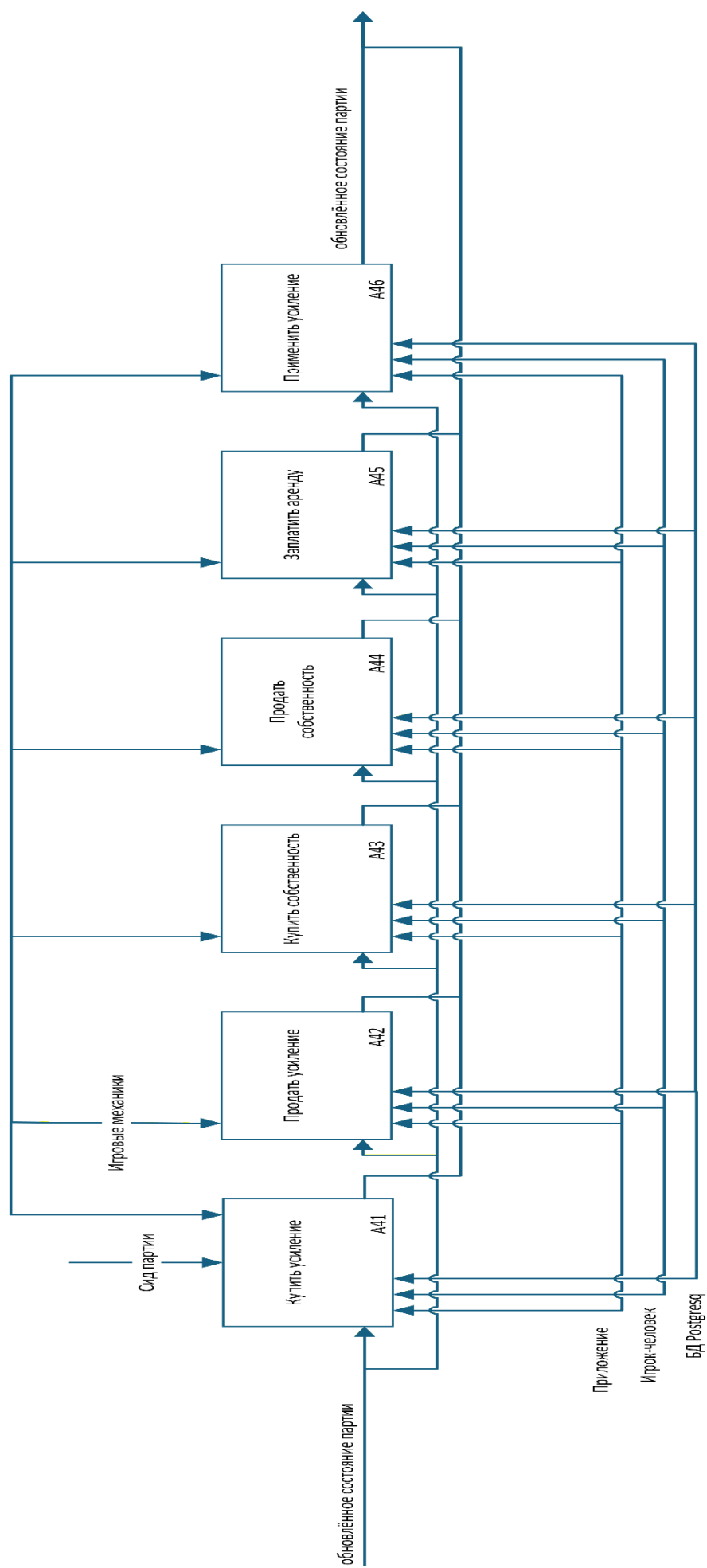


Рисунок Г.2 — Декомпозиция чёрного ящика

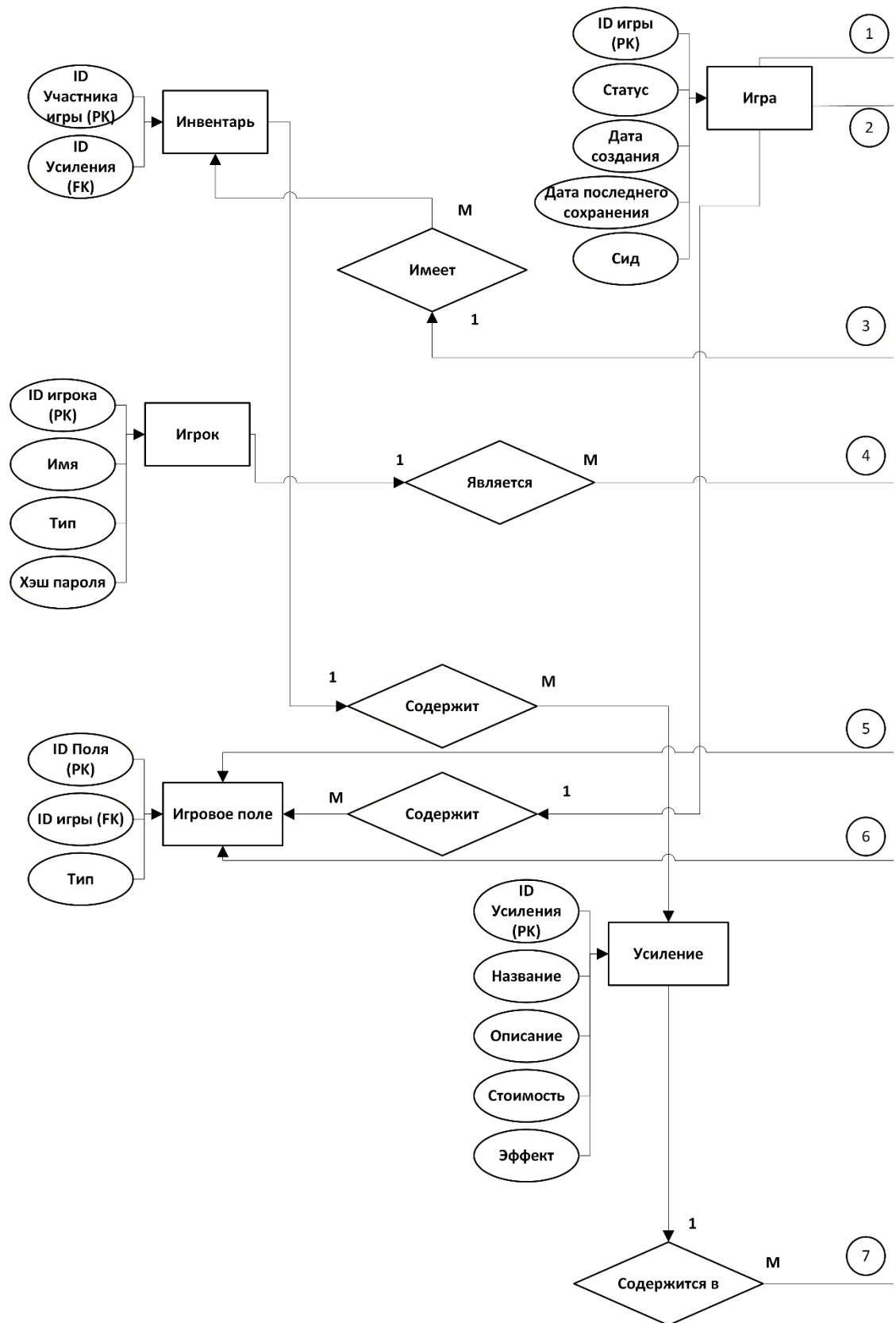


Рисунок Г.3 – Информационная модель

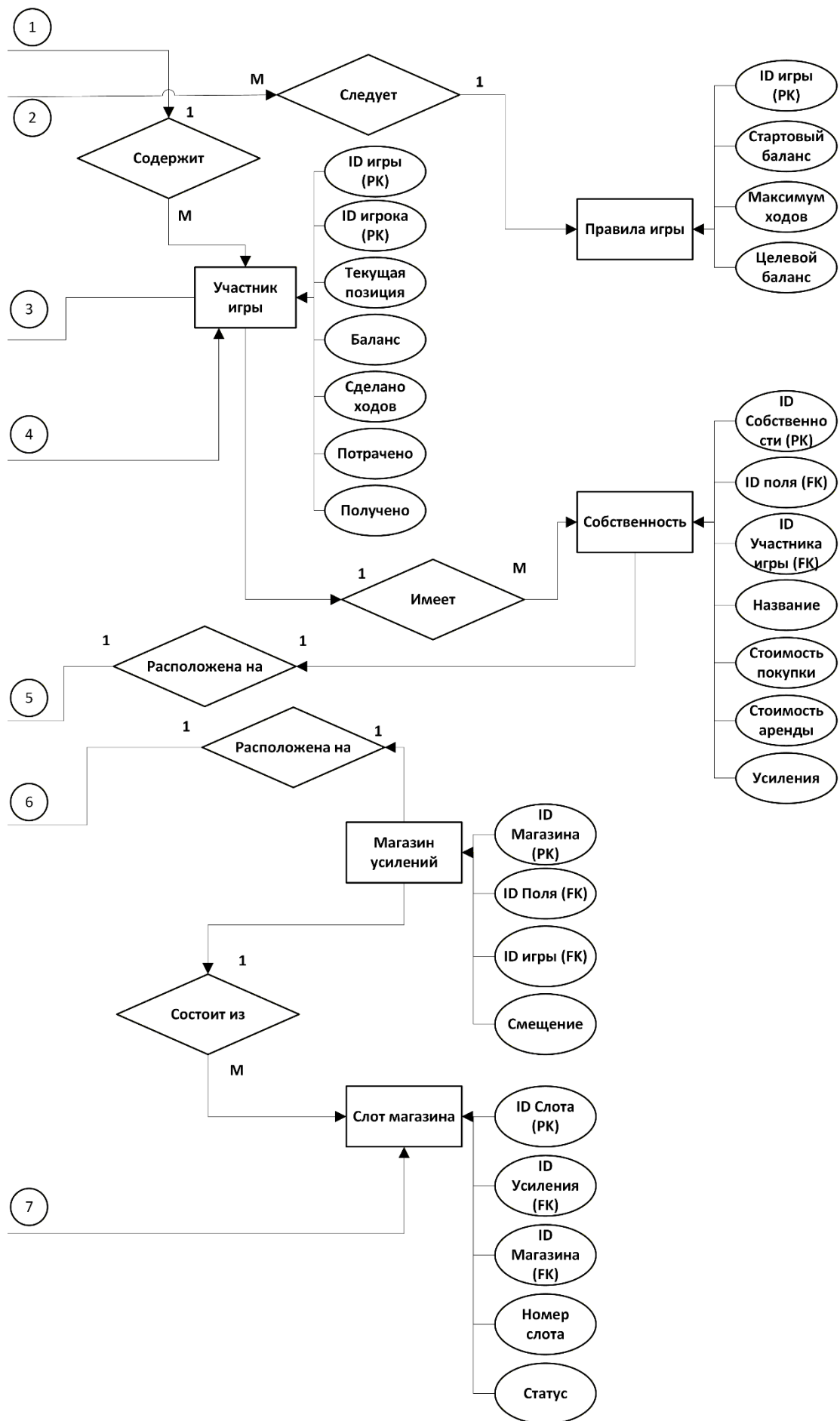


Рисунок Г.4 – Информационная модель

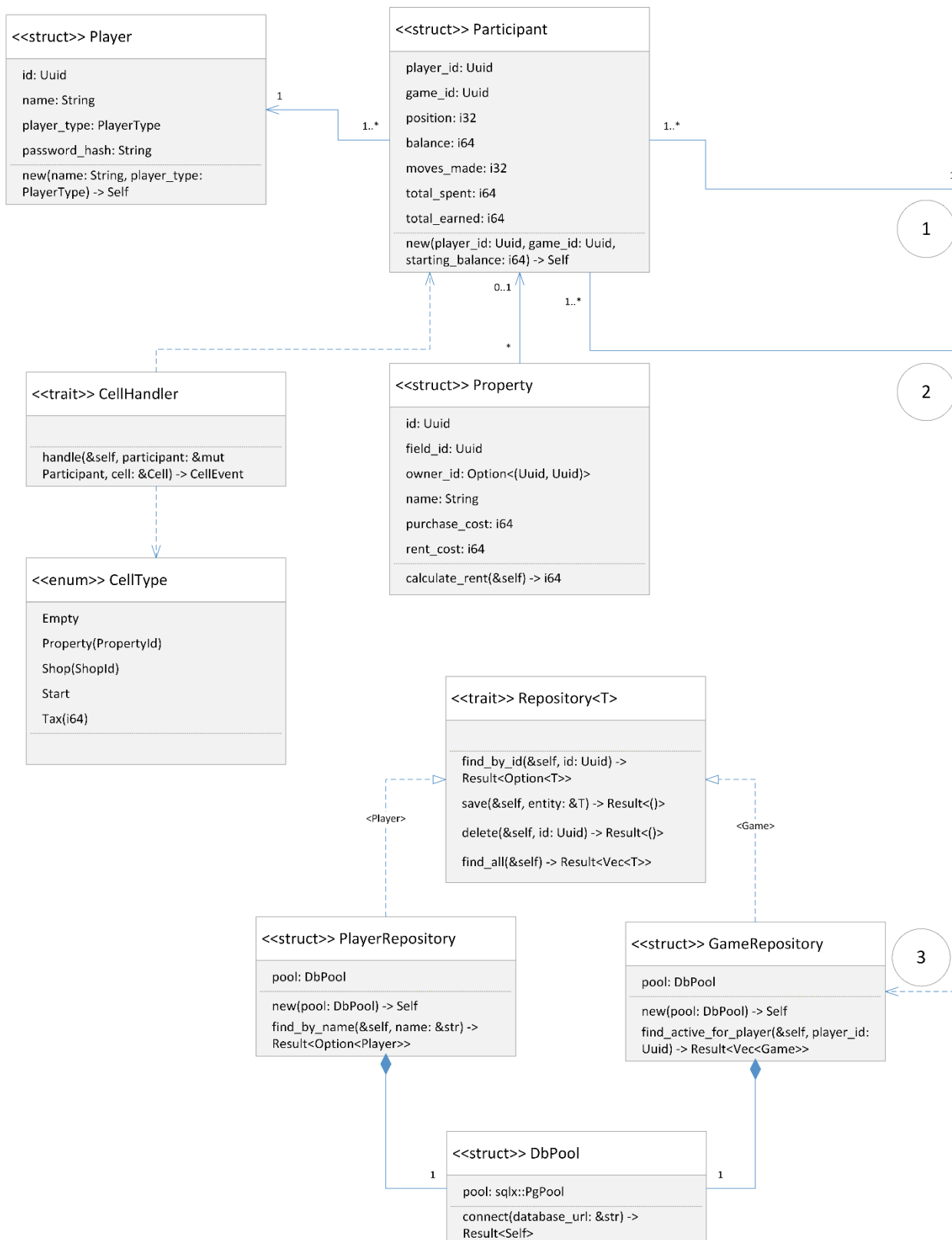


Рисунок Г.5 – Структурная модель

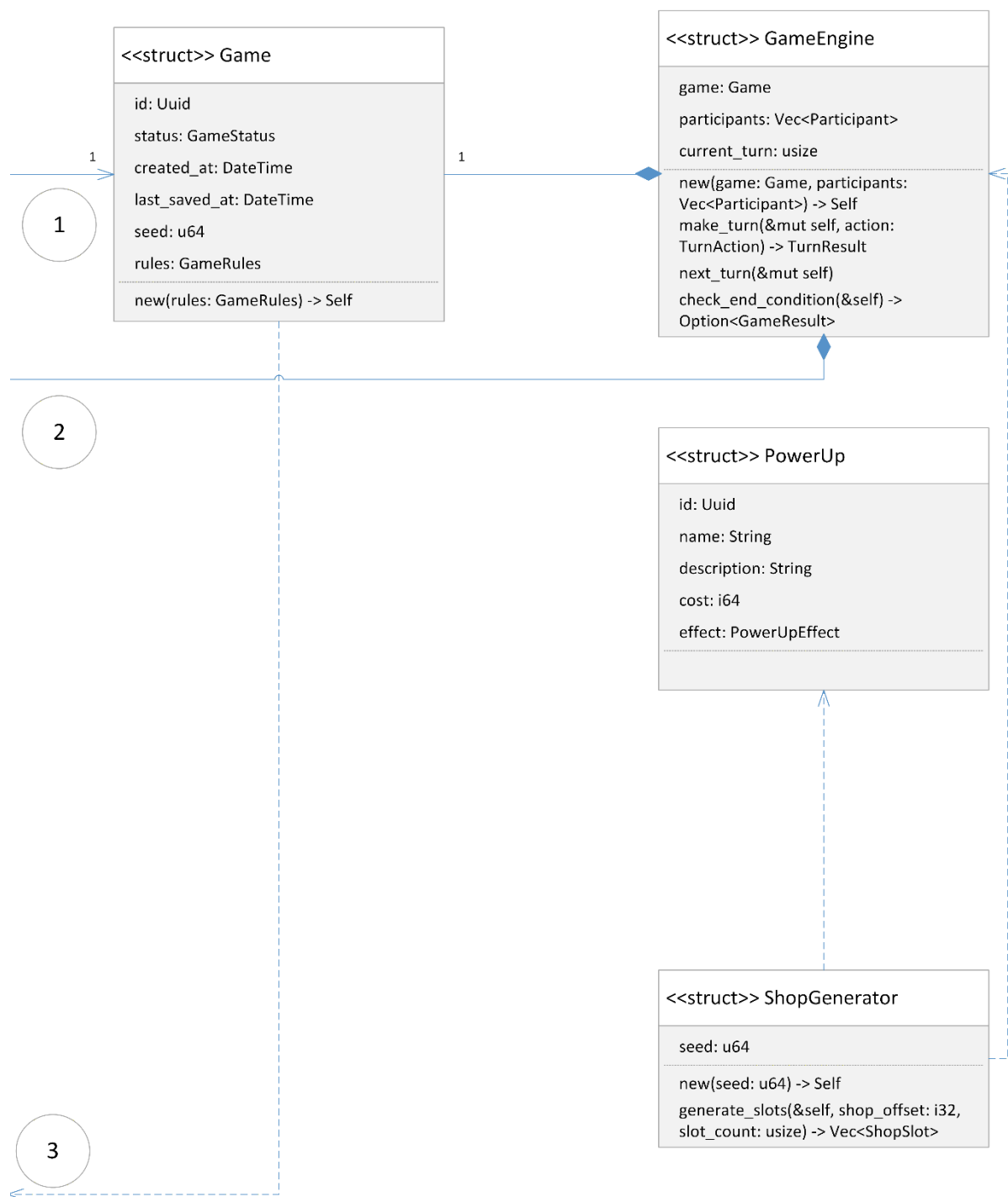


Рисунок Г.6 – Структурная модель

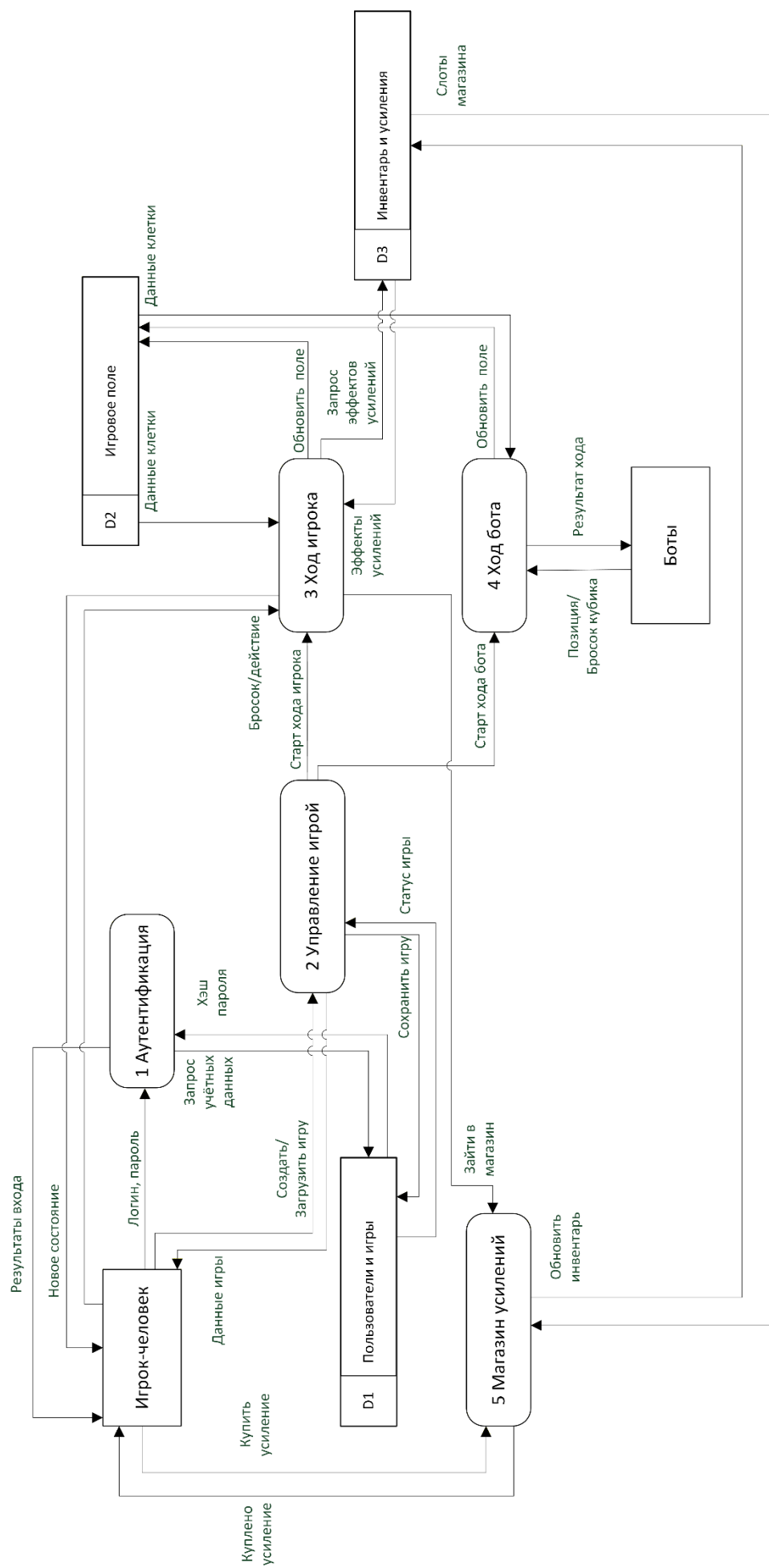


Рисунок Г.7 – Диаграмма потоков данных

Приложение Д
(Обязательное)
Ссылки на репозитории

<https://gitflic.ru/project/vegetablevictim/dreambreakerzov>

<https://github.com/MURUMokay/DreamBreaker/tree/main>

Приложение Е
(Обязательное)
Результаты ручного тестирования

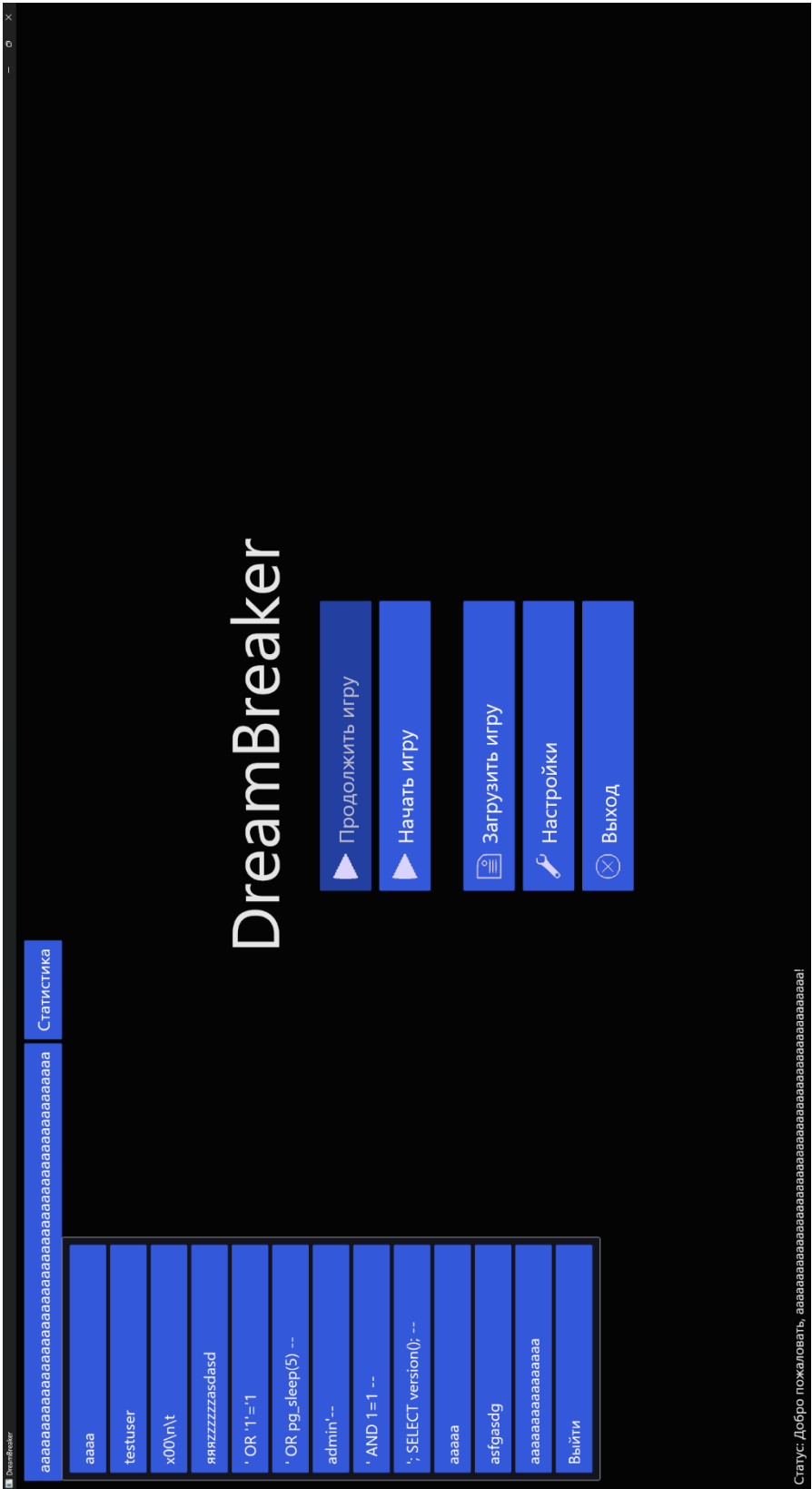


Рисунок Е.1 – Результат ручного тестирования на ввод SQL-Инъекции

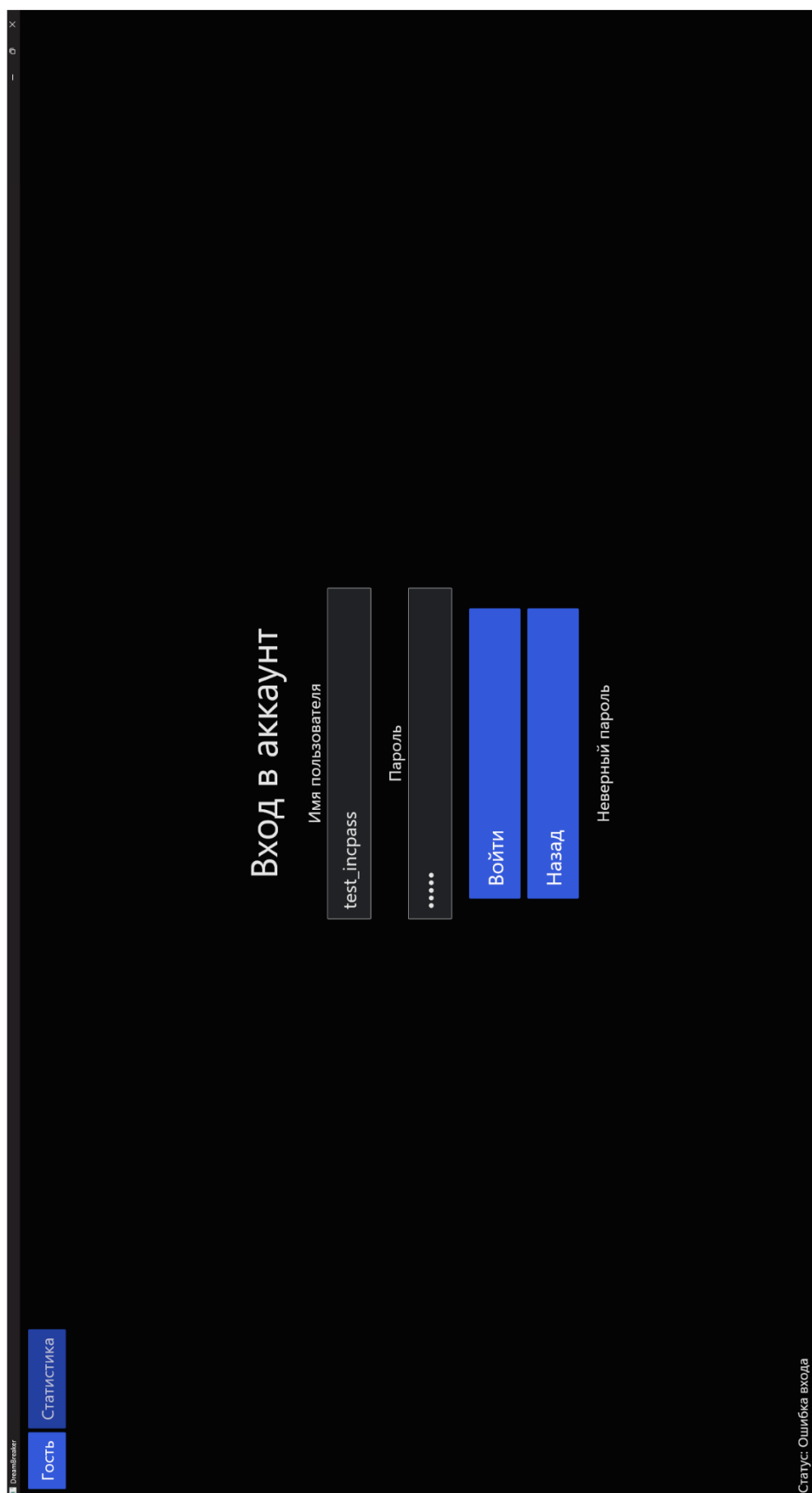


Рисунок Е.2 – Результат ручного тестирования на авторизацию с неверным паролем

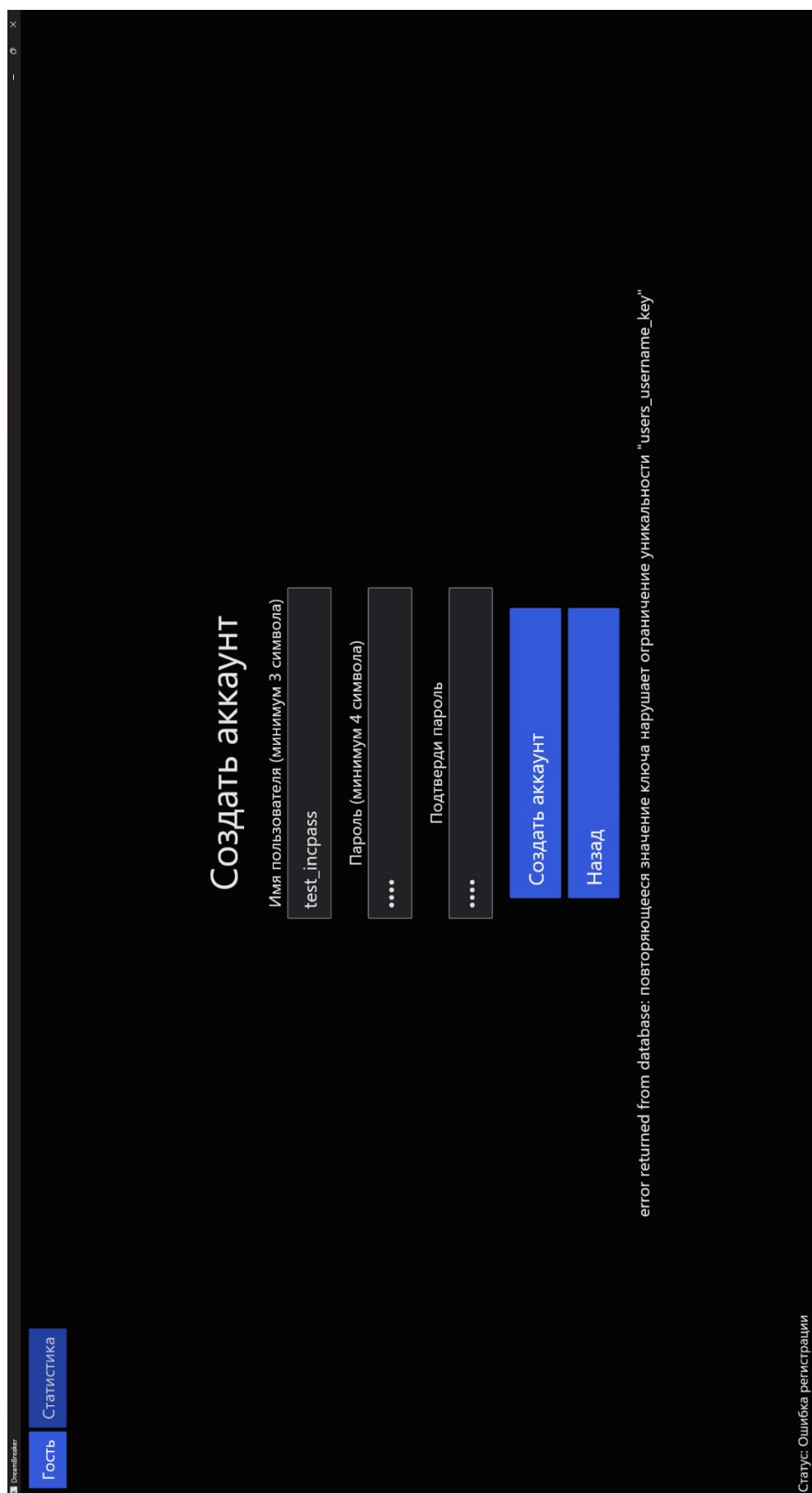


Рисунок Е.3 – Результат ручного тестирования на авторизацию с дубликатом
логина

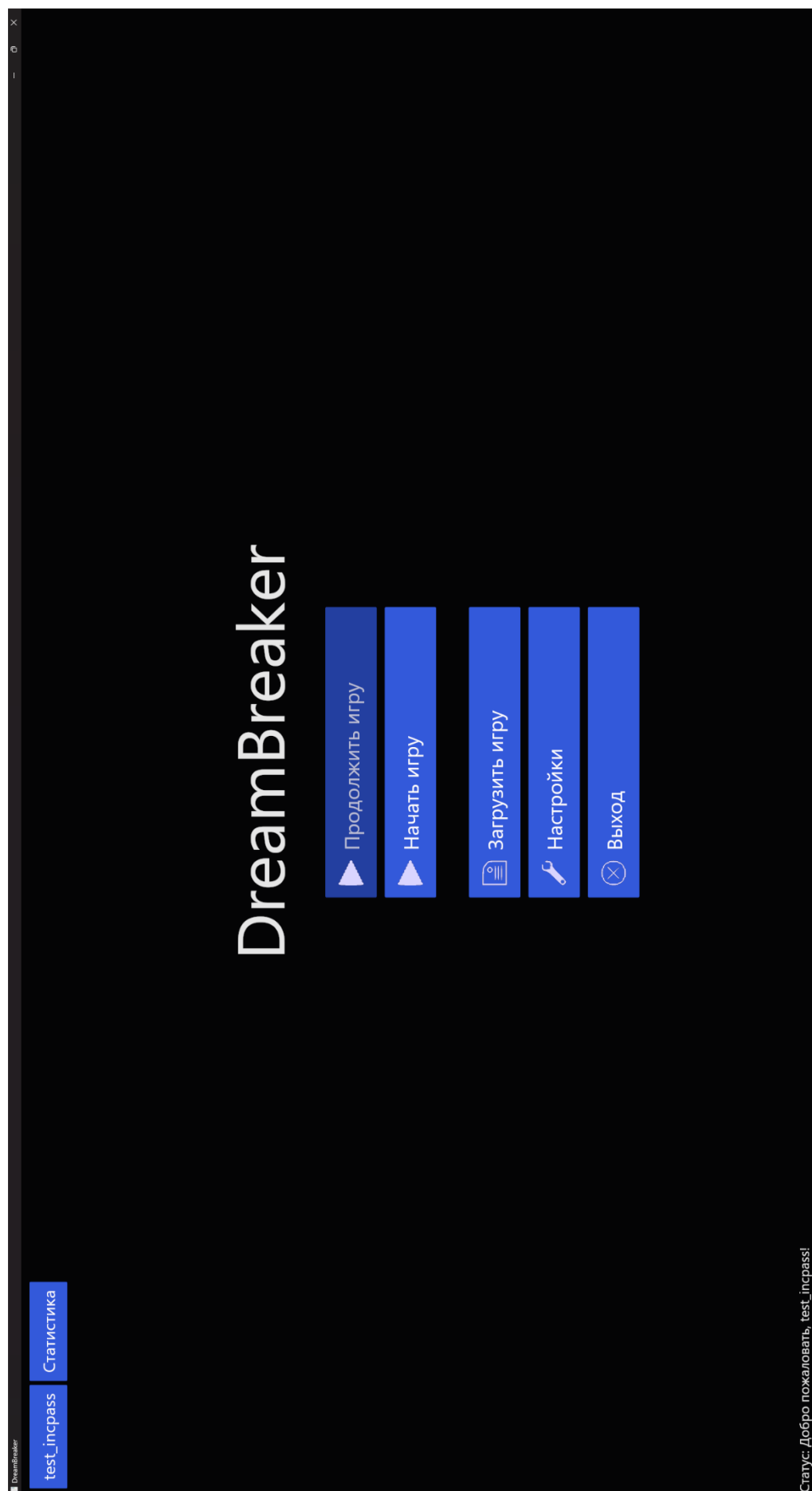


Рисунок Е.4 – Результат ручного тестирования на успешную авторизацию

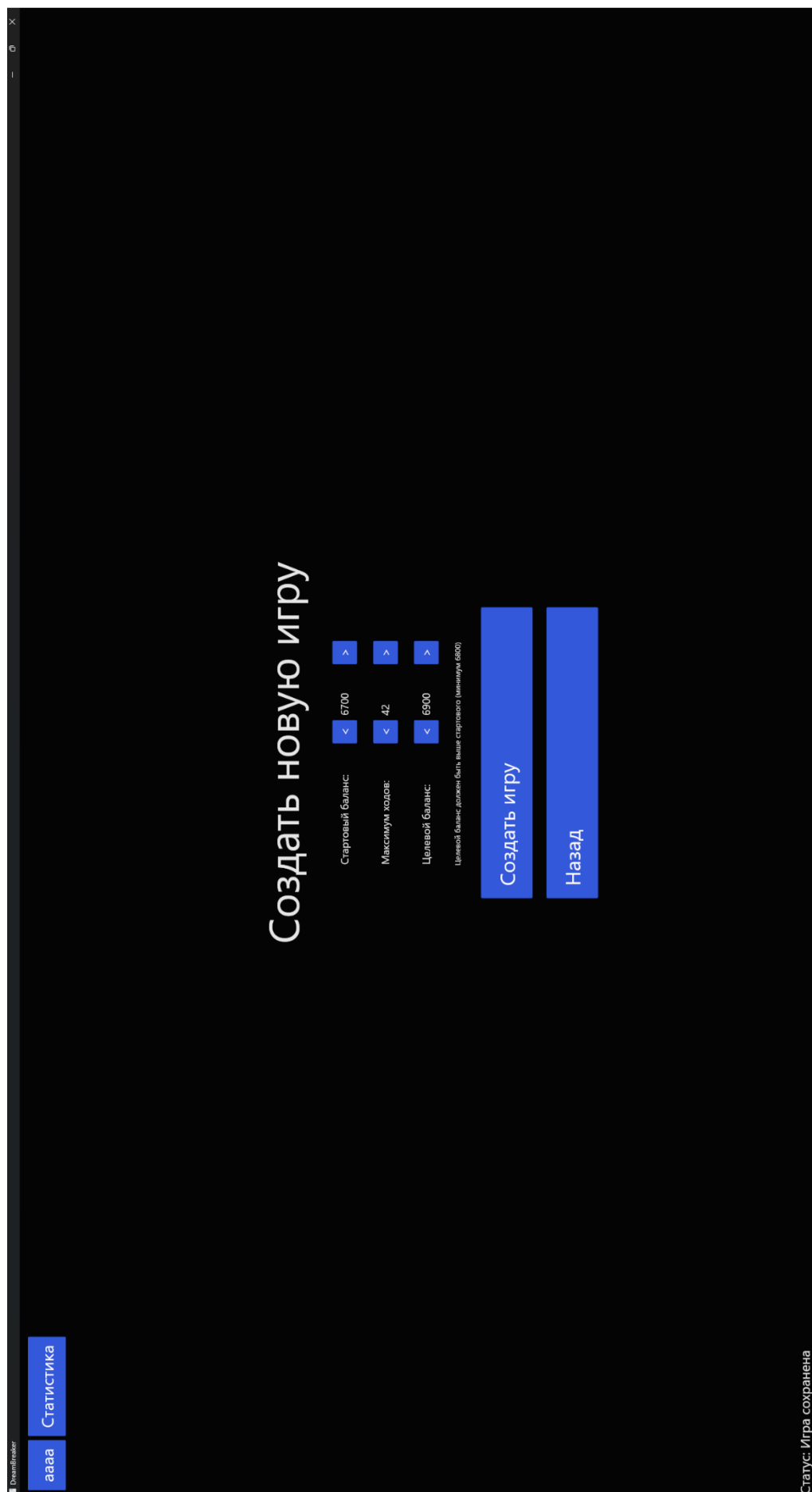


Рисунок Е.5 – Результат ручного тестирования на успешное создание игры

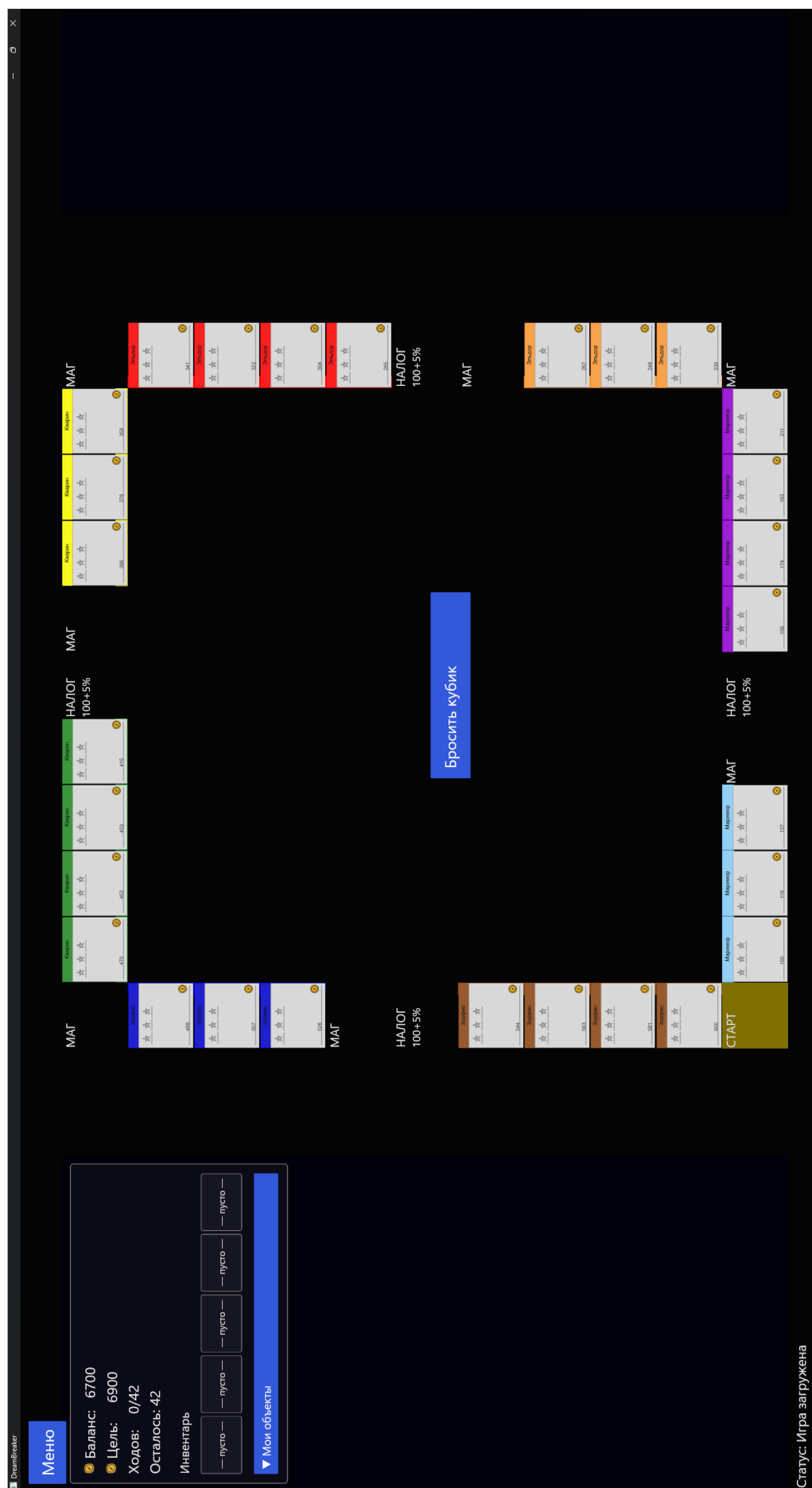


Рисунок Е.6 – Результат ручного тестирования на успешное создание игры



Рисунок Е.7 – Результат ручного тестирования на успешную загрузку игры

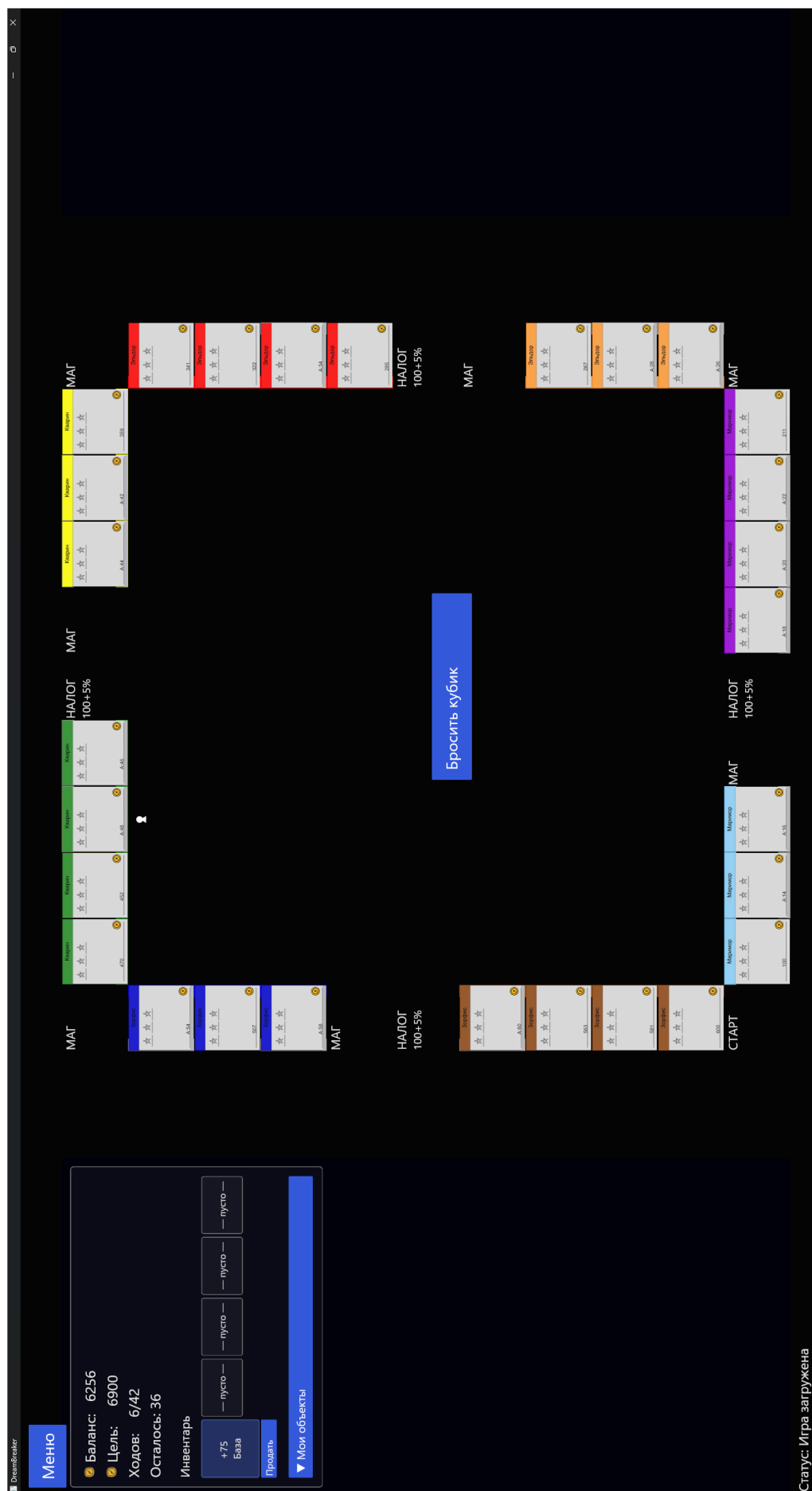


Рисунок Е.8 – Результат ручного тестирования на успешную загрузку игры